

University of Tripoli

Faculty of Engineering

Department of Computer Engineering

Reinforcement Learning for Adaptive Traffic Engineering in Software-Defined Network

A Graduation Project Report Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Engineering

Prepared by:

Maher Abdulnasir Alqadhi

Student ID: 2210249576

Supervised by:

Dr. Suad El-Geder

Spring 2026

CERTIFICATE OF APPROVAL

This is to certify that the graduation project entitled:

"Reinforcement Learning for Adaptive Traffic Engineering in Software-Defined Network"

submitted by **Maher Abdulnasir Alqadhi** (Student ID: **2210249576**) in partial fulfillment of the requirements for the degree of **Bachelor of Science in Computer Engineering** has been approved by the examining committee after public defense and examination.

Supervisor:
Dr. Suad El-Geder Signature: .....

Internal Examiner:
Prof./Dr. ..... Signature: .....

External Examiner:
Prof./Dr. ..... Signature: .....

Head of Department:
Department of Computer Engineering, University of Tripoli Signature: .....

DECLARATION OF ACADEMIC INTEGRITY

I, the undersigned **Maher Abdulnasir Alqadhi**, hereby declare that this graduation project report entitled **"Reinforcement Learning for Adaptive Traffic Engineering in Software-Defined Network"** is the outcome of my own original research and engineering work conducted under the academic supervision of **Dr. Suad El-Geder** at the Department of Computer Engineering, Faculty of Engineering, University of Tripoli.

I affirm that:

1. All conceptual designs, mathematical derivations, software algorithms, neural network implementations, simulation harnesses, and experimental evaluation suites presented in this document were developed by me, except where explicit bibliographic reference and citation are made to published academic literature.

2. No portion of this work has been submitted previously for any academic degree, diploma, or professional qualification at this university or any other institution of higher learning.

3. All sources of technical literature, software libraries, and data sets utilized in the realization of this project have been fully acknowledged in the bibliography in accordance with academic standards.

Maher Abdulnasir Alqadhi
Tripoli, Libya
Spring 2026

. . . . . .
Candidate Signature

DEDICATION

To my beloved parents, whose unconditional love, sacrifices, prayers, and steadfast belief in my potential have been the guiding light throughout my life and academic journey;

To my brothers and sisters, for their unwavering encouragement, patience, and companionship;

To all my esteemed professors and teachers at the University of Tripoli, who generously shared their knowledge, wisdom, and passion for engineering excellence;

And to every passionate researcher and student striving to harness technology and artificial intelligence for the advancement of society.

ABSTRACT

Uneven traffic loads can congest some network links while other routes remain available. This graduation project develops a software-defined networking prototype that selects routes using current link information and a trained reinforcement-learning agent. The work includes a routing controller, a network-state manager, a web dashboard, automated test scripts, and an active/standby controller pool. Python, Ryu, OpenFlow 1.3, Open vSwitch, Mininet, and PyTorch are used to implement and test the system. The routing agent uses a dueling double deep Q-network to choose from up to four candidate paths, with a reward that considers utilisation, delay, jitter, loss, and hop count. Evaluation consists of a software-model comparison across five network topologies, 120 packet-emulation trials on Tree and Fat-Tree networks, and separate controller-recovery tests. In the final software-model run, mean sampled path-bottleneck utilisation decreases from 66.80% for the OSPF-style baseline to 30.80% for the learned policy, while modelled delay decreases from 158.34 to 44.50 ms. Packet-emulation results are less uniform: on Tree TCP traffic, mean throughput is 34.08 Mbps for the learned policy, compared with 28.56 Mbps for shortest-path routing and 38.09 Mbps for least-loaded routing. The project therefore provides a working platform for studying adaptive routing and shows that its usefulness depends on the topology and workload. Its main limitations are software-only evaluation, a small candidate-path set, and a packet-test schedule that does not maintain background traffic throughout every measurement.

ACKNOWLEDGEMENT

First and foremost, all praise, gratitude, and adoration are due to Allah Almighty, the Most Merciful and Most Gracious, for granting me the perseverance, health, intellectual clarity, and strength to complete this graduation project report and reach this major academic milestone.

I wish to express my deepest, most sincere gratitude and utmost respect to my academic supervisor, **Dr. Suad El-Geder**, for her exceptional mentorship, invaluable guidance, continuous encouragement, and rigorous academic critique throughout the conception, development, and evaluation of this research. Her technical expertise in telecommunications and computer networking, along with her unyielding insistence on scientific precision, provided the cornerstone for overcoming critical engineering obstacles and improving the rigor of this work.

I also extend my sincere appreciation to the distinguished faculty members and technical staff of the **Department of Computer Engineering, Faculty of Engineering at the University of Tripoli**. I am deeply indebted for the comprehensive foundational education, rigorous laboratory training, and inspiring scholarly environment they provided throughout my undergraduate curriculum.

No words can adequately convey my profound debt of gratitude to my beloved parents; to my father for his lifelong sacrifices, wisdom, and steadfast belief in my potential, and to my mother for her unconditional love, endless prayers, and constant moral support. My heartfelt appreciation also goes to my brothers and sisters for their continuous encouragement and patience during the demanding phases of this project.

Finally, I warmly thank my fellow colleagues, classmates, and friends at the University of Tripoli who accompanied me along this demanding academic path, sharing fruitful technical debates, collaborative problem solving, and steadfast camaraderie.

Maher Abdulnasir Alqadhi

TABLE OF CONTENTS

Abstract i

Acknowledgement ii

List of Figures vii

List of Tables ix

List of Abbreviations x

1 Introduction 1

1.1 Project Background and Problem 1
1.2 Project Aim and Objectives 1
1.3 Importance of the Project 2
1.4 Method and Main Project Steps 2
1.5 Project Scope and Deliverables 2
1.6 Report Organisation 3

2 Background 5

2.1 Development and Test Environment 5
2.2 Software Tools and Reasons for Selection 5
2.3 Software-Defined Networking and Routing 5
2.4 Learning-Based Route Selection 7
2.5 Experience Replay and Prioritized Sampling 8
2.6 Understanding the Evaluation Metrics 9
2.7 Controller Recovery 10
2.8 Chapter Summary 10

3 Related Work 11

3.1 Recent Learning-Based Routing Systems 11
3.1.1 ENERO 11
3.1.2 Graph-Based Routing Agents 11
3.1.3 RouteNet-Fermi 11
3.2 Earlier Systems and Routing Baselines 11
3.3 How Related Work Informed the Project 12
3.4 Chapter Summary 12

4 Project Design and Implementation **14**
4.1 Design . . . 14
4.1.1 Requirements and Implemented Modules . . . 14
4.1.2 Implementation Considerations . . . 14
4.1.3 Top-Level Closed-Loop Control Architecture . . . 15
4.1.4 Continuous 10-Dimensional State Space Representation and Action Masking . . . 16
4.1.5 Candidate Action Space Pruning via 4-Tier Hybrid Diversity Routing . . . 17
4.1.6 D3QN Neural Network Architecture . . . 19
4.1.7 Implemented Multi-Objective Reward Function . . . 19
4.1.8 Binary SumTree Prioritized Experience Replay Architecture . . . 20
4.1.9 Health-Checked Active/Standby Multi-Controller Pool . . . 21
4.1.10 OpenFlow 1.3 Group Tables and Data-Plane Protection . . . 22
4.1.11 Closed-Loop Control Execution Timeline . . . 23
4.2 Experiments . . . 24
4.2.1 Evaluation Environment and Testbed Provisioning . . . 24
4.2.2 Benchmark Network Topologies . . . 24
4.2.3 Software-Model Traffic Generation . . . 25
4.2.4 Routing Baselines . . . 26
4.3 Testing and Evaluation . . . 26
4.3.1 Key Performance Metrics . . . 26
4.3.2 Test Procedures and Recorded Data . . . 27
4.3.3 Automated Test Suite and Functional Verification . . . 27
4.3.4 Checks Against the Project Objectives . . . 28
4.4 Results and Discussion . . . 28
4.4.1 Final Software-Model Routing Comparison . . . 28
4.4.2 Latency, Jitter, and Packet Loss Distributions . . . 30
4.4.3 Final Mininet Packet-Test Results . . . 31
4.4.4 Traffic Load Scaling and Congestion Transition . . . 33
4.4.5 Model-Seed and Workload Checks . . . 34
4.4.6 Checks of Policy Mode and Network Components . . . 36
4.4.7 Stress-Test Results . . . 37
4.4.8 Transfer Checks on Unseen Random Topologies . . . 39
4.4.9 Comparison with a Small Optimisation Reference . . . 39
4.4.10 Active/Standby Controller Pool Failover Benchmarks . . . 41
4.4.11 Practical Limitations . . . 42
4.4.12 Limits on Interpreting the Results . . . 43
4.4.13 Chapter Summary . . . 43

5 Conclusion **44**
5.1 Work Completed . . . 44
5.2 Main Results and Usefulness . . . 44
5.3 Limitations . . . 44

6 Future Work **46**
6.1 Improve the Packet-Test Procedure . . . 46
6.2 Make Installation and Reproduction Easier . . . 46
6.3 Improve Route Selection and Comparison . . . 46
6.4 Extend Controller-Recovery Testing . . . 47
6.5 Evaluate Additional Forwarding Platforms . . . 47

References **48**

A Mathematical Formulations and Proofs **50**
A.1 Identifiability of the Dueling Architecture . . . 50

B OpenFlow 1.3 Flow Table, Group Table, and REST API Specifications **51**
B.1 OpenFlow 1.3 Instruction Pipeline Architecture . . . 51
B.2 OpenFlow 1.3 Flow Table Entry Structure . . . 51
B.3 OpenFlow 1.3 Group Table Specifications . . . 51
B.4 REST API Endpoints . . . 53

C Automated Test Suite Verification Listing **54**

D Supporting Implementation Notes **56**
D.1 Candidate Paths and Input Features . . . 56
D.2 Replay Priorities and Storage . . . 56
D.3 Queue-Model Limits . . . 56
D.4 Meaning of the Recovery Measurements . . . 56
D.5 Supporting Files . . . 57

E User & Administrator Deployment Manual and Operational Runbook **58**
E.1 System Prerequisites and Dependencies . . . 58
E.1.1 Hardware Requirements & Reference Deployment Environment . . . 58
E.1.2 Software Requirements . . . 58
E.2 Installation and Environment Initialization . . . 59
E.2.1 Step-by-Step Installation . . . 59
E.3 Automated Execution Runbook . . . 60
E.3.1 1. Execute Unit & Integration Tests (42 Tests) . . . 60
E.3.2 2. Execute the Head-to-Head Tournament Benchmark . . . 60

E.3.3 3. Execute Multi-Topology Stress Testing 60
E.3.4 4. Evaluate Zero-Shot Random Graph Generalization 60
E.3.5 5. Launch the Prototype (Controller + Web Dashboard) 60
E.3.6 6. Standalone Web Dashboard Launcher 60
E.4 Interactive Web Dashboard User Guide 61
E.5 REST API Specification & Endpoint Reference 62
E.6 Mininet CLI Operations and OVS Diagnostics 64
E.6.1 Reproducible Packet-Level Mininet Experiment 65
E.6.2 OpenFlow Flow Table Inspection via Open vSwitch 65
E.7 Pareto Sensitivity Ablation and Dynamic Telemetry Tuning 66
E.7.1 Configuring Shadow Reservations and TCAM Allocation 66
E.7.2 Operational Troubleshooting and Recovery Runbook 67

LIST OF FIGURES

4.1 Closed-Loop SDN Control Plane and DRL Application Architecture. 15
4.2 Loopless Candidate Route Synthesis via Yen's K-Shortest Paths Algorithm. . . 17
4.3 Dueling Double Deep Q-Network (D3QN) Architecture. 19
4.4 Prioritized Experience Replay (PER) Binary SumTree Data Structure. 20
4.5 3-Node Active/Standby Controller Pool and Failover Stage Timeline. 21
4.6 End-to-End Control-Loop Telemetry and Inference Execution Timeline. 23
4.7 Conceptual Architectural Layout of Benchmark Datacenter Fabrics: (a) Fat-Tree Clos (schematic representation illustrating 4 core, 4 aggregation, and 4 edge switches for visual clarity; the full evaluated $k = 4$ fabric deploys 20 switches and 16 hosts); (b) Spine-Leaf Fabric (schematic showing 3 spines and 4 leaves; the full evaluated tournament fabric models 4 spines, 8 leaves, and 32 hosts). 25
4.8 Multi-Dimensional Routing Tournament Radar Performance Comparison Across Algorithms. 29
4.9 Cumulative Distribution Functions (CDFs) of End-to-End Latency and Jitter Across Protocols. 30
4.10 Measured Mininet condition means (120 trials): `iperf` throughput and UDP datagram loss across policies and topologies. Note: TCP retransmissions were uninstrumented in the test harness; ICMP echo loss is reported separately as an out-of-band diagnostic of concurrent reachability under saturation and must not be conflated with TCP transport-layer packet loss. Detailed bootstrap confidence intervals and controller overhead diagnostics appear in Tables 4.8 and 4.9. 32
4.11 Traffic Load Scaling Curves Demonstrating Modelled Congestion Response. . . 33
4.12 D3QN Training Convergence Curves Showing Mean Episode Reward and TD Error Loss. 34
4.13 Multi-Seed Paired Performance Effects and Statistical Confidence Intervals. . . 35
4.14 Policy-Mode Ablation Comparing Pure Neural D3QN Against Safety-Gated D3QN Across 25 Paired Workload-Seed Evaluations. 36
4.15 Bootstrap Confidence Distributions of Component Ablations Across Stages: Local Plain DQN, Double DQN, Dueling DDQN without PER, and Full D3QN. 37
4.16 Stress Testing Under Asymmetric Core Link Saturation and Synchronized High-Concurrency Bursts. 38
4.17 Stress Benchmarks Across Irregular and Complex Topologies. 38
4.18 Zero-Shot Transfer Learning Evaluation on Three Unseen Random Graphs (12-Node Erdős-Rényi, 14-Node Watts-Strogatz, and 16-Node Barabási-Albert). . . 39

4.19 Optimality Gap Between D3QN and the Exact HiGHS MILP Solver. 40
4.20 Panoramic View of the Real-Time Operational Web Telemetry Dashboard. . . . 41

E.1 VMware Virtual Machine Hardware Configuration for the Experimental Platform (Ubuntu 64-bit, 4 vCPUs, 8 GB RAM, 60 GB SCSI Storage). 59

LIST OF TABLES

1.1 Traceability Matrix Mapping Project Objectives to Dissertation Sections and Appendices. 4

2.1 Main Software Tools and Their Roles in the Project 6

3.1 Related Systems and Their Relevance to the Project 12
3.2 Comparative Taxonomy Contrasting EC499 with State-of-the-Art SDN TE and DRL Routing Systems. 13

4.1 Implemented Modules and Their Responsibilities 14
4.2 Continuous 10-Dimensional Normalized State Feature Vector Layout 16
4.3 OpenFlow 1.3 Group Table Specifications Implemented in the Prototype 23
4.4 D3QN Routing Agent and PER Hyperparameter Specifications 24
4.5 Tests and Recorded Outputs 27
4.6 Project Functions and Supporting Checks 28
4.7 Final Software-Model Results Across Five Topologies (Seed-42 Snapshot; see Section 4.4.5 for 125-Case Variance Decomposition) 29
4.8 Measured Transport Results: Means and 95% Bootstrap Intervals (10 Trials per Condition) 31
4.9 ICMP Diagnostics and Controller Decisions: Means and 95% Bootstrap Intervals 32

B.1 OpenFlow 1.3 Flow Table Entry Field Specifications 52
B.2 OpenFlow 1.3 Group Table Field Specifications Implemented in the Prototype . 52

E.1 REST API Reference: Telemetry, Diagnostics, and Model Inference Endpoints 63
E.2 REST API Reference: Dynamic Control, Traffic Injection, and Simulation Endpoints 64
E.3 Operational Troubleshooting and Anomaly Remediation Runbook 67

LIST OF ABBREVIATIONS

Abbreviation	Full Definition
AI	Artificial Intelligence
ANOVA	Analysis of Variance
API	Application Programming Interface
ARP	Address Resolution Protocol
ASIC	Application-Specific Integrated Circuit
BGP	Border Gateway Protocol
CDF	Cumulative Distribution Function
C-MDP	Constrained Markov Decision Process
CPU	Central Processing Unit
D3QN	Dueling Double Deep Q-Network
DDQN	Double Deep Q-Network
Dijkstra SPF	Dijkstra Shortest Path First
DQN	Deep Q-Network
DRL	Deep Reinforcement Learning
ECMP	Equal-Cost Multi-Path
FIFO	First-In First-Out
FlowMod	OpenFlow Flow Modification Message
GNN	Graph Neural Network
GPU	Graphics Processing Unit
HA	High Availability
HTTP	Hypertext Transfer Protocol
IETF	Internet Engineering Task Force
INT	In-Band Network Telemetry
IP	Internet Protocol
IS-IS	Intermediate System to Intermediate System
JSON	JavaScript Object Notation
KSP	K-Shortest Paths
LLDP	Link Layer Discovery Protocol
LLR	Least Loaded Routing
MARL	Multi-Agent Reinforcement Learning
MDP	Markov Decision Process
MILP	Mixed-Integer Linear Program
MPLS	Multiprotocol Label Switching

Abbreviation	Full Definition
MTU	Maximum Transmission Unit
OFP	OpenFlow Protocol
OSPF	Open Shortest Path First (RFC 2328)
OVS	Open vSwitch
P4	Programming Protocol-Independent Packet Processors
PER	Prioritized Experience Replay
QoS	Quality of Service
Raft	Consensus Algorithm for Distributed Replicated State Machines
REST	Representational State Transfer
RFC	Request for Comments
RL	Reinforcement Learning
RSVP-TE	Resource Reservation Protocol with Traffic Engineering Extensions
Ryu	Component-Based Software-Defined Networking Framework
SDN	Software-Defined Networking
SLA	Service Level Agreement
TCAM	Ternary Content-Addressable Memory
TCP	Transmission Control Protocol
TD	Temporal Difference
TE	Traffic Engineering
UDP	User Datagram Protocol
UMCF	Unsplittable Multi-Commodity Flow
VIP	Virtual Internet Protocol Address
VLAN	Virtual Local Area Network
WAN	Wide Area Network
WSP	Widest Shortest Path

CHAPTER 1 INTRODUCTION

1.1 Project Background and Problem

Traffic engineering concerns the way network traffic is assigned to available routes. A shortest-path policy is straightforward to configure, but its link costs do not necessarily reflect changing traffic loads. Several transfers can therefore compete for the same link even when another route is available. The resulting queues can increase delay and packet loss. OSPF and equal-cost multipath forwarding provide established routing mechanisms, but their configured costs alone do not describe current queue occupancy or utilisation (Moy, 1998).

Software-defined networking provides a practical way to investigate alternative routing decisions. An application can collect switch counters, maintain a topology model, and install forwarding rules through a controller. The problem addressed in this project is how to implement such a system, use a trained routing policy within it, and determine when its decisions are useful compared with simpler routing methods.

1.2 Project Aim and Objectives

The aim is to design, implement, and evaluate a software-defined traffic-engineering prototype that selects routes using network-state information and deep reinforcement learning.

The project objectives are to:

1. Build a network-state manager that maintains topology, link statistics, candidate paths, and flow information.
2. Implement a routing agent that chooses a valid path from a bounded candidate set.
3. Integrate the agent with a Ryu/OpenFlow controller and provide a dashboard for inspecting the system.
4. Implement comparable shortest-path, equal-cost, widest-path, and least-loaded baselines for software-model testing.
5. Test the software and compare routing behaviour using model-based experiments and separate Mininet packet measurements.
6. Provide a health-checked active/standby pool and test client HTTP recovery when an endpoint fails.

1.3 Importance of the Project

Route selection affects how effectively a network uses its existing links. An adaptive routing method may reduce congestion without increasing link capacity, but it also adds computation, training, and monitoring requirements. Comparing the learned policy with deterministic heuristics is therefore necessary: a more complicated method is useful only if its behaviour justifies that complexity.

The project also provides a teaching and testing platform. The dashboard makes route choices visible, while the scripts and saved results allow a future student to inspect the implementation and repeat the evaluation. The intended users are students and engineers investigating routing in a controlled software environment.

1.4 Method and Main Project Steps

The work was organised around implementation followed by functional and performance testing:

1. Create the topology and state-management modules, including candidate-path generation and link-load accounting.
2. Implement the dueling double deep Q-network and its prioritised replay buffer using PyTorch.
3. Connect routing decisions to the Ryu controller's flow-rule installation path.
4. Add the dashboard, routing baselines, and active/standby endpoint pool.
5. Run the automated software checks and the five-topology software-model comparison.
6. Execute the separate 120-trial Mininet experiment, retain its raw outputs, and analyse throughput, UDP datagram loss, ICMP diagnostics, and decision time.

Chapter 4 describes the final implementation and test procedures. The full source is supplied separately; supporting commands and specifications are included in the appendices.

1.5 Project Scope and Deliverables

The completed deliverables are the Python implementation, trained model, controller application, web dashboard, automated tests, experiment scripts, saved data, and this report. The evaluation covers five software-model fixtures and packet emulation on Tree and Fat-Tree networks.

The routing policy selects one of up to four loopless candidate paths. It uses ten summary features, rather than a complete graph representation. The controller pool supports local endpoint selection and HTTP retry; it does not maintain a replicated consensus log. The platform was tested on a virtualised Linux host. Hardware-switch performance, production availability, and generalisation to arbitrary networks are outside the demonstrated scope.

1.6 Report Organisation

The report follows the department's graduation-project template:

- Chapter 2 introduces the tools and concepts needed to understand the implementation and explains why they were chosen.
- Chapter 3 reviews relevant earlier systems and recent work that informs the design.
- Chapter 4 describes the project under Design, Experiments, Testing and Evaluation, and Results and Discussion.
- Chapter 5 summarises what was achieved, the usefulness of the system, and its limitations.
- Chapter 6 identifies practical next steps based on those limitations.

References identify the external material used. The appendices contain supporting equations, API specifications, the test listing, implementation notes, and the operating manual.

Table 1.1 provides an explicit traceability matrix mapping each of the six project objectives defined in Section 1.2 to its corresponding design section, experimental evaluation, and technical appendix.

Table 1.1: Traceability Matrix Mapping Project Objectives to Dissertation Sections and Appendices.

Objective	**Core Engineering Scope**	**Design & Implementation**	**Evaluation & Appendices**
Obj. 1	Network state manager & 4-tier candidate routing	Section 4.1	Section 4.3, App. B
Obj. 2	D3QN routing agent with SumTree PER	Section 4.1	Section 4.4, App. A
Obj. 3	Ryu OpenFlow controller & real-time dashboard	Section 4.1	Section 4.3, App. E
Obj. 4	Baseline routing heuristics (SPF, ECMP, WSP, LLR, MILP)	Section 4.1, 4.2	Section 4.4
Obj. 5	5-topology software tournament & 120-run Mininet emulation	Section 4.2	Section 4.4.3, App. C
Obj. 6	Active/standby controller pool & transparent HTTP failover	Section 4.1	Section 4.4, App. D

CHAPTER 2 BACKGROUND

This chapter introduces the equipment, software, and routing concepts needed to understand the project. It explains their role in the implementation; detailed program behaviour and test results are presented in Chapter 4.

2.1 Development and Test Environment

The project uses a Linux virtual machine for development and network emulation. A virtual machine makes the setup accessible without a physical switch laboratory, although processor scheduling and shared host resources can influence timings. The supplied VMware configuration is shown in Appendix E. Software versions for the packet experiment are taken from its saved environment manifest rather than inferred from the current machine.

2.2 Software Tools and Reasons for Selection

Table 2.1 summarizes the software packages, libraries, and protocols employed across the control, data, and experimental planes, detailing their specific architectural roles and selection rationales.

Two Python environments are used because the legacy Ryu/Eventlet stack and the scientific tools have different compatibility requirements. The packet runner uses the scientific environment with access to the system Mininet bindings; the controller is started using its separate Python 3.10 environment. This separation is important when reproducing the project.

2.3 Software-Defined Networking and Routing

In software-defined networking, the controller computes forwarding decisions and the switches apply installed rules. For this project, the controller obtains topology information through discovery events and derives link rates from differences between successive byte-counter readings. It then chooses an output path and sends OpenFlow rule updates.

The comparison includes an OSPF-style inverse-capacity cost baseline, hop-count shortest-path routing, equal-cost multipath routing, widest-shortest-path routing, and least-loaded routing. These provide useful reference behaviours. The software-model OSPF-style baseline implements route costs rather than the full distributed OSPF protocol (Moy, 1998). The packet experiment uses the separate hop-count SPF and least-loaded policies.

For a directed link e, utilisation is the offered load divided by capacity:

$$u_e = \frac{\lambda_e}{C_e}. \tag{2.1}$$

Table 2.1: Main Software Tools and Their Roles in the Project

Tool	Project role	Reason for selection
Python	Controller logic, experiments, and analysis	Allows the controller and learning components to share data structures.
Ryu	OpenFlow controller application	Provides packet parsing, switch events, and access to OpenFlow messages.
OpenFlow 1.3	Communication with switches	Supports forwarding-rule installation and port/flow counter requests (McKeown et al., 2008; Open Networking Foundation, 2012).
Open vSwitch	Software forwarding plane	Runs with Mininet and exposes flow tables and port counters.
Mininet	Packet-emulation testbed	Runs Linux network stacks and application traffic in network namespaces.
PyTorch	Neural-network implementation	Supports training, model saving/loading, and CPU inference.
NetworkX	Topology and path processing	Provides graph structures and shortest-path routines.
NumPy and Matplotlib	Data analysis and figures	Support numerical summaries and reproducible plots.
iperf and ping	Packet-test measurements	Supply transport throughput/UDP statistics and separate ICMP diagnostics.

Here both λ_e and C_e are expressed in Mbps. Values above one represent overload in the software model. Each traffic demand is assigned a single candidate path:

$$\sum_{p \in \mathcal{P}_k} x_{k,p} = 1, \qquad x_{k,p} \in \{0,1\}. \tag{2.2}$$

A capacity-feasible assignment would also require

$$\sum_{k} \sum_{p : e \in p} d_k x_{k,p} \leq C_e. \tag{2.3}$$

The learned reward encourages lower congestion but does not enforce these inequalities as hard admission constraints. The implemented candidate paths and reward are described in the design section.

2.4 Learning-Based Route Selection

Reinforcement learning formalizes optimal sequential decision-making within the framework of a Markov Decision Process (MDP) defined by the 5-tuple $\mathcal{M} = \langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{S}$ represents the state space, $\mathcal{A}$ is the action space, $\mathcal{P}(s' \mid s,a) = \mathbb{P}(S_{t+1} = s' \mid S_t = s, A_t = a)$ defines the environmental transition dynamics, $\mathcal{R}(s,a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ is the immediate scalar reward function, and $\gamma \in [0,1)$ is the temporal discount factor (Sutton and Barto, 2018). Under policy $\pi(a \mid s)$, the expected return from state-action pair (s,a) is denoted by the action-value function $Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \;\middle|\; S_t = s, A_t = a\right]$. The optimal policy π^* satisfies the fundamental Bellman Optimality Equation:

$$Q^*(s,a) = \mathcal{R}(s,a) + \gamma \sum_{s' \in \mathcal{S}} \mathcal{P}(s' \mid s,a) \max_{a' \in \mathcal{A}} Q^*(s',a'). \tag{2.4}$$

In high-dimensional state spaces where tabular dynamic programming is intractable, Deep Q-Networks (DQN) approximate $Q^*(s,a)$ with a parameterized neural network $Q(s,a;\theta)$ (Mnih et al., 2015). The standard single-DQN optimizes the parameter vector θ by minimizing the mean squared Bellman error against a slowly varying target network parameterized by θ^-:

$$Y_t^{\mathrm{DQN}} = R_{t+1} + \gamma \max_{a' \in \mathcal{A}} Q(S_{t+1}, a'; \theta^-). \tag{2.5}$$

However, single-DQN incurs an upward maximization bias due to the Jensen-type inequality $\mathbb{E}[\max_a (Q(s,a) + \varepsilon_a)] \geq \max_a \mathbb{E}[Q(s,a)]$ when evaluation noise ε_a is present. To prevent systematic overestimation of candidate path values, Double DQN decouples greedy action selection from action evaluation (Van Hasselt et al., 2016):

$$Y_t^{\mathrm{DDQN}} = R_{t+1} + \gamma Q\left(S_{t+1}, \arg\max_{a' \in \mathcal{A}} Q(S_{t+1}, a'; \theta); \theta^-\right). \tag{2.6}$$

Terminal transitions omit the discounted next-state term. Rather than performing abrupt periodic hard updates, the target parameters θ^- track the online parameters θ continuously via a Polyak soft-update rule (Polyak-Rupert tracking) with soft-update parameter $\tau \ll 1$ ($\tau = 0.005$):

$$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-. \tag{2.7}$$

The dueling network architecture further improves policy evaluation by factoring the scalar state-action value into separate state-value $V(s)$ and state-dependent action-advantage $A(s,a)$ streams (Wang et al., 2016). To guarantee identifiability—ensuring that $V(s)$ accurately reflects the state value without shifting arbitrary constants between streams—the advantage outputs are centralized by subtracting their discrete mean:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s,a';\theta,\alpha) \right). \tag{2.8}$$

A complete mathematical proof of this identifiability condition and gradient uniqueness is provided in Appendix A.

2.5 Experience Replay and Prioritized Sampling

Replay stores previous transitions so they can be reused during training, breaking temporal autocorrelation across consecutive state transitions. Uniform experience replay selects transitions uniformly at random ($P(i) = 1/N$), which dilutes rare high-error transitions—such as transient link-saturation and buffer-overflow events—amid abundant nominal samples. Prioritized Experience Replay (PER) addresses this limitation by sampling transitions proportionally to their Temporal Difference (TD) error magnitude (Schaul et al., 2016).

For transition $i = (S_i, A_i, R_i, S_i')$, the double-DQN TD error is formulated as:

$$\delta_i = R_i + \gamma Q\left(S_i', \arg\max_{a'} Q(S_i', a'; \theta); \theta^-\right) - Q(S_i, A_i; \theta). \tag{2.9}$$

The transition priority p_i is computed with a small positive constant $\varepsilon_{\text{per}} = 0.01$ to ensure non-zero sampling probability, and an exponent $\alpha \in [0,1]$ determining the degree of prioritization:

$$p_i = \left(|\delta_i| + \varepsilon_{\text{per}}\right)^{\alpha}. \tag{2.10}$$

When $\alpha = 0$, sampling collapses to the uniform baseline; $\alpha = 0.6$ is deployed in the prototype. The discrete sampling probability distribution across buffer capacity N is defined as:

$$P(i) = \frac{p_i}{\sum_{k=1}^{N} p_k} = \frac{(|\delta_i| + \varepsilon_{\text{per}})^{\alpha}}{\sum_{k=1}^{N} (|\delta_k| + \varepsilon_{\text{per}})^{\alpha}}. \tag{2.11}$$

Because prioritized sampling alters the empirical data distribution, it introduces estimation bias.

This bias is corrected via Importance Sampling (IS) weights w_i, scaled by the buffer capacity N and normalized by the maximum weight in the mini-batch to preserve gradient scale:

$$w_i = \left(\frac{1}{N} \cdot \frac{1}{P(i)}\right)^{\beta}, \qquad \tilde{w}_i = \frac{w_i}{\max_j w_j}, \tag{2.12}$$

where $\beta \in [0,1]$ is annealed linearly from initial value $\beta_0 = 0.4$ toward 1.0 at the conclusion of training:

$$\beta_t = \min\left(1.0,\, \beta_0 + t \cdot \Delta\beta\right). \tag{2.13}$$

The network is trained by minimizing the weighted Smooth L1 (Huber) loss over mini-batch size $B = 32$:

$$\mathcal{L}(\theta) = \frac{1}{B}\sum_{i=1}^{B} \tilde{w}_i \cdot \ell_{\delta_0}(\delta_i), \quad \text{where } \ell_{\delta_0}(\delta_i) = \begin{cases} \frac{1}{2}\delta_i^2 & \text{if } |\delta_i| \le \delta_0, \\ \delta_0\left(|\delta_i| - \frac{1}{2}\delta_0\right) & \text{otherwise,} \end{cases} \tag{2.14}$$

with transition threshold $\delta_0 = 1.0$. The project implements the priority prefix sums using a binary SumTree data structure, enabling $O(\log N)$ priority updates and sampling operations.

2.6 Understanding the Evaluation Metrics

The software model estimates queueing effects with an analytical finite-buffer $M/M/1/K$ queueing model applied across each directed link $e \in E$. Let λ_e denote the Poisson arrival rate and μ_e denote the exponential service capacity, yielding link traffic intensity $\rho = \lambda_e/\mu_e = u_e$. Under finite buffer capacity $K = 100$ packets, the steady-state buffer-overflow loss probability P_K is formulated as:

$$P_K = \begin{cases} \dfrac{(1-\rho)\rho^K}{1-\rho^{K+1}} & \text{if } \rho \neq 1, \\ \dfrac{1}{K+1} & \text{if } \rho = 1. \end{cases} \tag{2.15}$$

The expected number of packets residing in the queueing subsystem L is given by:

$$L = \frac{\rho}{1-\rho} - \frac{(K+1)\rho^{K+1}}{1-\rho^{K+1}}, \tag{2.16}$$

and applying Little's Law with effective arrival rate $\lambda_{\text{eff}} = \lambda_e(1-P_K)$ yields the mean queue waiting time W_q and total link traversal delay $D(e) = d_{\text{prop}}(e) + W$, where total time in the queueing system is $W = W_q + \frac{1}{\mu_e} = \frac{L}{\lambda_{\text{eff}}}$:

$$W_q = \frac{L}{\lambda_e(1-P_K)} - \frac{1}{\mu_e}, \qquad D(e) = d_{\text{prop}}(e) + \frac{L}{\lambda_e(1-P_K)}, \tag{2.17}$$

where $d_{\text{prop}}(e)$ represents physical link propagation latency. Because real packet networks exhibit heavy-tailed burst distributions, these analytical formulas serve as controlled simulation estimates rather than direct ASIC measurements.

The Mininet experiment records a different set of observations. Iperf reports transport throughput and, for UDP, datagram loss and jitter. Ping reports round-trip time and ICMP echo loss. TCP retransmission loss is not recorded. In particular, ICMP echo loss must not be interpreted as TCP packet loss, and RTT must not be labelled one-way delay. RFC 3393 provides background on packet delay variation (Demichelis and Chimento, 2002); the model's bounded jitter proxy is not a direct implementation of that measured statistic.

2.7 Controller Recovery

The project also includes a pool that probes controller endpoints and retries client requests through an available standby. Its purpose is to examine client HTTP recovery after endpoint failure. Local selection bookkeeping, HTTP recovery, and OpenFlow reconnection are different operations and require different measurements. The pool does not implement a distributed replicated log or establish a production availability target.

2.8 Chapter Summary

The tools provide the components needed for a working software prototype: graph processing, a trained route selector, controller integration, a software forwarding plane, and repeatable tests. The main evaluation requirement is to keep model estimates, packet observations, and controller-recovery timings clearly identified. The next chapter explains how related systems influenced these design choices.

CHAPTER 3 RELATED WORK

This chapter reviews systems relevant to the project's route selector, controller integration, and evaluation. Recent work is considered first, followed by the earlier routing methods used as references. The purpose is to explain the design choices, rather than to claim that this prototype replaces those systems.

3.1 Recent Learning-Based Routing Systems

3.1.1 ENERO

ENERO combines a graph-based reinforcement-learning policy with a local-search stage to improve WAN routing configurations (Almasan et al., 2022a). Its two-stage design is relevant because it shows that a learned decision can be combined with a conventional optimisation method. The present project instead selects one of four candidate paths using a fixed summary vector. This is simpler to integrate with the controller, but retains less information about the graph.

3.1.2 Graph-Based Routing Agents

Almasan et al. combine graph neural networks with reinforcement learning for an optical-routing use case (Almasan et al., 2022b). Their work evaluates transfer to unseen topologies. It informs the choice to examine transfer separately in this project, while also identifying a limitation of the ten-feature representation. The current implementation uses path summaries and does not implement a graph neural network; its transfer results therefore apply only to the sampled graphs and workloads.

3.1.3 RouteNet-Fermi

RouteNet-Fermi predicts network delay, jitter, and loss using a graph neural network (Ferriol-Galmés et al., 2023). It is a performance model rather than the same route-selection task. Its relevance is the need to represent topology, traffic, and queue behaviour accurately. The present project uses a simpler analytical queue model and checks it against separate packet-emulation observations. Results from the two evaluation methods are reported separately.

3.2 Earlier Systems and Routing Baselines

RL-Routing studies reinforcement-learning routing with throughput and delay rewards and compares it with OSPF and least-loaded routing (Chen et al., 2020). This supports the inclusion of simple routing baselines in the project. Stampa et al. also investigate learned routing

configurations for reducing delay (Stampa et al., 2017). The local plain-DQN ablation is a project implementation baseline, not a reproduction of either publication.

Earlier SDN systems demonstrate other ways to allocate traffic. Hedera schedules flows in data-centre networks (Al-Fares et al., 2010), while B4 and SWAN use centralised optimisation for WAN traffic allocation (Jain et al., 2013; Hong et al., 2013). They provide context for controller-based traffic management, but differ from the small virtual testbed and per-flow route selector built here.

The project also compares with shortest-path and equal-cost forwarding methods (Moy, 1998). These baselines are useful because they are inexpensive and provide predictable reference behaviour. Least-loaded routing provides a further check on whether the trained model offers a practical benefit over a load-aware heuristic.

3.3 How Related Work Informed the Project

Table 3.1 relates the reviewed systems to the final implementation.

Table 3.1: Related Systems and Their Relevance to the Project

System or approach	Relevant feature	Project design decision
ENERO (2022)	Learned routing combined with local search	Compare the neural policy with a separate heuristic-assisted mode.
DRL with GNNs (2022)	Graph-based representation and transfer tests	Keep the present input small and state its transfer limits.
RouteNet-Fermi (2023)	Learned prediction of network performance	Distinguish the analytical model from packet measurements.
RL-Routing (2020)	Throughput/delay rewards and routing comparisons	Include SPF and least-loaded baselines.
Hedera, B4, and SWAN	Controller-based traffic allocation	Use a controller to install routes and collect counters.

To systematically situate this dissertation within the broader traffic-engineering landscape, Table 3.2 presents a comparative taxonomy contrasting the EC499 platform against representative classical SDN systems, centralized optimization frameworks, and deep reinforcement learning routing architectures across six key architectural dimensions: algorithmic methodology, state observation representation, action space structure, control plane substrate, loop/forwarding safety mechanisms, and empirical evaluation fidelity.

3.4 Chapter Summary

The reviewed work supports the use of network-state information, reproducible comparisons, and explicit transfer tests. The project brings these ideas together in a smaller software pro-

Table 3.2: Comparative Taxonomy Contrasting EC499 with State-of-the-Art SDN TE and DRL Routing Systems.

System	Algorithm	State Representation	Action Space	Substrate	Safety Mechanism	Evaluation Fidelity
Hedera (Al-Fares et al., 2010)	Demand-fit heuristic	Host flow rates	Discrete path set	OpenFlow	Flow pre-reservation	Hardware & Mininet
B4 / SWAN (Jain et al., 2013; Hong et al., 2013)	Centralized LP / TE	Tunnel traffic matrices	Continuous split ratios	OpenFlow / BGP	Bandwidth reservations	Production WAN hardware
Stampa et al. (Stampa et al., 2017)	Single DQN	Traffic matrix vector	Discrete path index	OpenFlow / Ryu	None (unconstrained)	OMNeT++ discrete-event sim
RL-Routing (Chen et al., 2020)	Deep DRL (DDPG)	Link utilization vector	Continuous flow fractions	OpenFlow	Boundary clipping	Mininet network emulation
ENERO (Almasan et al., 2022a)	DRL + Local Search	Graph (Message-Passing GNN)	Discrete link weights	P4 / OpenFlow	Local search validation	OMNeT++ packet simulation
RouteNet-Fermi (Ferriol-Galmés et al., 2023)	Graph Neural Net	Graph topology & queues	Performance inference	SDN Telemetry	GNN calibration	Hardware testbed & sim
EC499 (This Work)	**Dueling D3QN + SumTree PER**	**10-D graph-invariant summary vector**	**Discrete $K = 4$ loopless candidate paths**	**Ryu OpenFlow 1.3 (Flow/-Group)**	**Precomputed simple paths & shadow reservation**	**500-step sim + 120-run Mininet emulation**

totype with controller integration, a dashboard, and separate recovery tests. The next chapter describes the implemented modules, the experiment procedures, and the final results.

CHAPTER 4 PROJECT DESIGN AND IMPLEMENTATION

This chapter documents the completed system and its tests. It follows the department template through four sections: **Design** (Section 4.1), **Experiments** (Section 4.2), **Testing & Evaluation** (Section 4.3), and **Results and Discussion** (Section 4.4).

4.1 Design

4.1.1 Requirements and Implemented Modules

The design translates the project objectives into cooperating modules. Routing must obtain network-state information, generate valid candidates, select a route, and install forwarding rules. The dashboard and saved logs allow that behaviour to be inspected. Table 4.1 identifies the main files.

Table 4.1: Implemented Modules and Their Responsibilities

Module	Responsibility
`controller/main_controller.py`	Receives switch events and packet-in messages, coordinates discovery, and requests statistics.
`controller/state_manager.py`	Maintains topology, host locations, link counters, path candidates, and software-model flow state.
`agent/dqn_router.py`	Defines networks, action selection, training updates, and checkpoint handling.
`agent/prioritized_replay.py`	Stores transitions and samples them using cumulative priorities.
`controller/policy_selector.py`	Provides the common pure and heuristic-assisted selection interface.
`controller/routing_module.py`	Converts selected routes into OpenFlow forwarding rules.
`controller/web_dashboard.py`	Provides telemetry, inspection, and software-model controls.
`controller/ha_controller_pool.py`	Probes endpoints and retries client requests through a standby.

4.1.2 Implementation Considerations

The controller and scientific tools use separate Python environments because their dependencies differ. The runner preserves the virtual-environment interpreter path so Ryu can import its required packages. A short unscored transport stream precedes the ping measurements, allowing host learning and reactive rule installation. Its output is retained alongside the measured data.

Dashboard controls that manipulate software-model state are distinct from operations on the live Mininet forwarding plane. HTTP recovery is also tested separately from routing-policy performance; experimental fast-failover groups are disabled in the packet comparison.

4.1.3 Top-Level Closed-Loop Control Architecture

The proposed platform operates as an autonomous closed-loop control system structured across three functional planes: the **Data Plane**, the **Control Plane**, and the **Intelligence Plane**. Figure 4.1 illustrates the top-level architectural layout and the continuous interaction cycle.

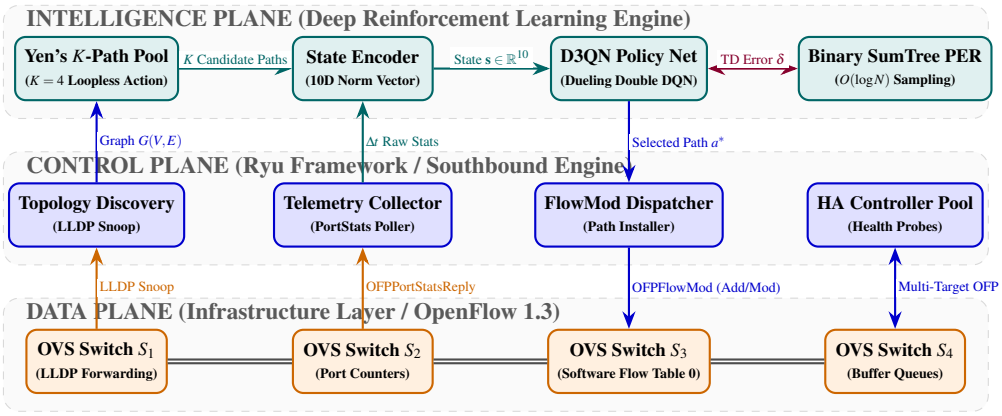


Figure 4.1: Closed-Loop SDN Control Plane and DRL Application Architecture.

The closed loop operates continuously:

1. Open vSwitch (OVS) switches in the Data Plane execute packet forwarding via software flow tables.

2. The Ryu controller polls port counters at a configurable interval (3.0 s by default in `config.py` for Mininet packet runs, or 1.0 s during high-frequency telemetry evaluation) using `OFPPortStatsRequest`.

3. First-order differentials Δtx_bytes are computed to derive instantaneous link load $\lambda_e(t)$ and utilization $u_e(t) = \lambda_e(t)/C_e$.

4. The State Encoder synthesizes a normalized 10-dimensional telemetry vector $\mathbf{s}_t \in \mathbb{R}^{10}$.

5. The PyTorch D3QN engine performs a forward inference pass (0.24 ms) to select a candidate loopless path $a_t^* = \arg\max_a Q(\mathbf{s}_t, a)$.

6. The FlowMod Dispatcher generates `OFPFlowMod` messages instructing switches to program corresponding forwarding rules.

7. Experiences $(\mathbf{s}_t, a_t, r_t, \mathbf{s}_{t+1}, d_t)$ are stored in the binary SumTree PER buffer for asynchronous training.

4.1.4 Continuous 10-Dimensional State Space Representation and Action Masking

Rather than using raw unscaled counters, the agent consumes a continuous 10-dimensional state vector $\mathbf{s} \in \mathbb{R}^{10}$, normalized into the numerical range $[0.0, 1.0]$. The vector explicitly encodes topological hop distance, per-candidate bottleneck utilizations, per-candidate end-to-end latencies, and global network congestion. The exact feature layout produced by `controller/state_manager.py` is formalized in Table 4.2.

Table 4.2: Continuous 10-Dimensional Normalized State Feature Vector Layout

Index	Feature Symbol	Physical Description	Normalization Mapping
s_0	$\hat{H}_{\min}$	Shortest path hop count (Dijkstra SPF)	$\min(1.0, \max(0.1, \text{hops}/10.0))$
s_1	$\hat{u}_0$	Bottleneck link utilization of Candidate Path 0	$\min(1.0, \max(0.0, \max_{e \in P_0} u_e))$
s_2	$\hat{u}_1$	Bottleneck link utilization of Candidate Path 1	$\min(1.0, \max(0.0, \max_{e \in P_1} u_e))$
s_3	$\hat{u}_2$	Bottleneck link utilization of Candidate Path 2	$\min(1.0, \max(0.0, \max_{e \in P_2} u_e))$
s_4	$\hat{u}_3$	Bottleneck link utilization of Candidate Path 3	$\min(1.0, \max(0.0, \max_{e \in P_3} u_e))$
s_5	$\hat{D}_0$	Normalized end-to-end latency of Candidate Path 0	$\min\left(1.0, \max\left(0.0, \sum_{e \in P_0} d_e / 50.0 \text{ ms}\right)\right)$
s_6	$\hat{D}_1$	Normalized end-to-end latency of Candidate Path 1	$\min\left(1.0, \max\left(0.0, \sum_{e \in P_1} d_e / 50.0 \text{ ms}\right)\right)$
s_7	$\hat{D}_2$	Normalized end-to-end latency of Candidate Path 2	$\min\left(1.0, \max\left(0.0, \sum_{e \in P_2} d_e / 50.0 \text{ ms}\right)\right)$
s_8	$\hat{D}_3$	Normalized end-to-end latency of Candidate Path 3	$\min\left(1.0, \max\left(0.0, \sum_{e \in P_3} d_e / 50.0 \text{ ms}\right)\right)$
s_9	$\hat{u}_{\text{peak}}$	Network-wide peak link utilization across all edges	$\min(1.0, \max(0.0, \max_{e \in E} u_e))$

Handling Variable Candidate Path Counts and Invalid Action Masking: When the network topology does not yield $K = 4$ loopless paths between an ingress-egress pair (e.g., in sparse meshes where only two loopless paths exist), two complementary mechanisms ensure mathematical consistency:

1. **Penalty Feature Padding:** For any candidate index $i \geq$ candidate_count, the state manager sets $\hat{u}_i = 1.0$ and $\hat{D}_i = 1.0$. This ensures the input tensor maintains a fixed dimension $\mathbb{R}^{10}$ while providing an immediate penalty signal to the neural network.

2. **Dynamic Invalid-Action Masking:** In the policy network forward pass (`agent/dqn_router.py`), action selection is restricted to physically available candidate paths via dynamic tensor slicing:

$$a^* = \arg \max_{a \in \{0, \ldots, K_{\text{avail}} - 1\}} Q(\mathbf{s}, a), \tag{4.1}$$

   where $K_{\text{avail}} = \min(K, \text{candidate_count})$. In exploration mode ($\varepsilon$-greedy), uniform random selection is similarly constrained to $\{0, \ldots, K_{\text{avail}} - 1\}$. This guarantees that the agent never selects a non-existent candidate route.

Fixed Summary Path Features vs. True Graph Invariance: This 10-dimensional state representation operates on *summary path statistics* rather than raw graph adjacency matrices. This design choice enables the model to ingest paths from different topologies without changing neural network weights or tensor dimensions. However, this is distinct from true graph permutation equivariance (such as that offered by Message-Passing Graph Neural Networks); our zero-shot

generalization findings represent empirical transfer over the tested graph distributions, rather than a theoretical proof of arbitrary graph invariance.

4.1.5 Candidate Action Space Pruning via 4-Tier Hybrid Diversity Routing

To bound the number of selectable routes and guarantee loop-freedom, the agent's action space $\mathcal{A}$ is constrained to $K = 4$ loopless candidate paths. Rather than relying solely on Yen's K-shortest paths algorithm (Yen, 1971), the state manager (`controller/state_manager.py`) synthesizes a 4-tier hybrid candidate pool combining hop-count optimality, edge-disjoint path diversity, and dynamic non-linear load awareness. Figure 4.2 illustrates the underlying spur-search mechanism of Yen's algorithm utilized during candidate generation.

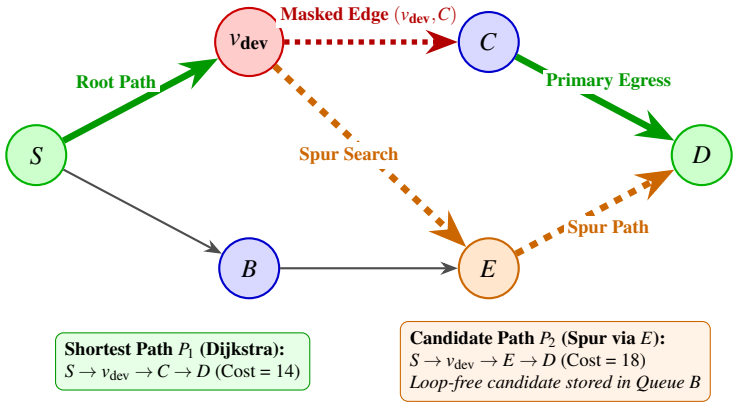


Figure 4.2: Loopless Candidate Route Synthesis via Yen's K-Shortest Paths Algorithm.

The 4-tier candidate routing pipeline computes selectable paths as follows:

1. **Tier 0 (P_0 — Shortest Path):** The global minimum-hop path computed via Dijkstra's algorithm:
$$P_0 = \text{Dijkstra}(G, s, t). \tag{4.2}$$

2. **Tier 1 (P_1 — Second Shortest Path):** The second loopless path $P_1 = \text{Yen}_2(G, s, t)$ produced by Yen's deviation search, providing immediate secondary topological proximity.

3. **Tier 2 (P_2 — Disjoint Diverse Bypass Path):** Selected from the set of loopless simple paths $\mathcal{P}_{\text{simple}}$ to minimize physical edge overlap with primary path P_0:
$$P_2 = \arg \min_{P \in \mathcal{P}_{\text{simple}} \setminus \{P_0, P_1\}} |E(P) \cap E(P_0)|, \tag{4.3}$$
breaking ties by path hop length. This guarantees an alternate detour path when primary links experience localized bottlenecks.

4. **Tier 3 (P_3 — Dynamic Load-Aware Path):** Recomputed on each routing invocation using Dijkstra's algorithm parameterized by a non-linear congestion cost function

$c_{\text{dynamic}}(e)$:

$$P_3 = \text{Dijkstra}(G, s, t; c_{\text{dynamic}}), \tag{4.4}$$

where for edge $e = (u, v)$ with base latency d_e and effective link utilization $u_{\text{eff}}(e)$:

$$c_{\text{dynamic}}(e) = d_e \cdot \left[1.0 + 25.0 \cdot (\max(0.0, u_{\text{eff}}(e) - 0.55))^2\right]. \tag{4.5}$$

Below 55% utilization, $c_{\text{dynamic}}(e)$ equals base latency d_e; above 55%, the cost increases quadratically, steering Tier 3 routing around congested core links.

Active Closed-Loop Shadow Reservation and Decay: Because OpenFlow port statistics polling occurs periodically (nominal interval $\Delta t = 3.0$ s in default Mininet configuration, configurable down to 1.0 s for fast evaluation), multiple elephant flows dispatched in rapid succession can pile up on the same egress link before physical telemetry reflects the increased utilization (micro-herd effect). To prevent this, the controller applies an instantaneous shadow bandwidth reservation $u_{\text{shadow}}(e,t)$ upon path installation, which decays exponentially over time according to a configured half-life $\tau_{1/2} = 1.5$ s:

$$u_{\text{shadow}}(e,t) = \sum_{f \in \mathcal{F}_e} \frac{R_f}{C_e} \cdot 2^{-\frac{t - t_{0,f}}{\tau_{1/2}}}, \tag{4.6}$$

where R_f is the estimated flow rate (nominal 15.0 Mbps), C_e is the link capacity, and $t_{0,f}$ is the flow arrival timestamp. The effective utilization consumed by the state encoder and dynamic path generator is thus:

$$u_{\text{eff}}(e,t) = u_{\text{meas}}(e,t) + u_{\text{shadow}}(e,t). \tag{4.7}$$

Weighted Cost Multi-Path (WCMP) Softmax Flow Distribution: When multipath distribution is enabled via OpenFlow 1.3 Select Group tables (`OFPGT_SELECT`), the routing module (`controller/routing_module.py`) calculates continuous traffic split proportions across candidate paths using a temperature-scaled Softmax over path quality scores:

$$\text{score}(p) = -\left(\beta_{\text{wcmp}} \cdot \max_{e \in p} u_e + 0.05 \cdot \sum_{e \in p} d_e\right), \tag{4.8}$$

where $\beta_{\text{wcmp}} = 4.0$. The normalized flow forwarding probability distribution $P(p)$ is formulated as:

$$P(p) = \frac{\exp\left(\text{score}(p) - \max_{p'} \text{score}(p')\right)}{\sum_{p'' \in \mathcal{P}} \exp\left(\text{score}(p'') - \max_{p'} \text{score}(p')\right)}. \tag{4.9}$$

These continuous probabilities are discretized into integer OpenFlow bucket weights $W(p) \in [1, 100]$:

$$W(p) = \max\left(1, \text{round}\left(100 \cdot P(p)\right)\right), \tag{4.10}$$

which are directly loaded into the OpenFlow switch bucket action list.

4.1.6 D3QN Neural Network Architecture

The D3QN neural network architecture is illustrated in Figure 4.3.

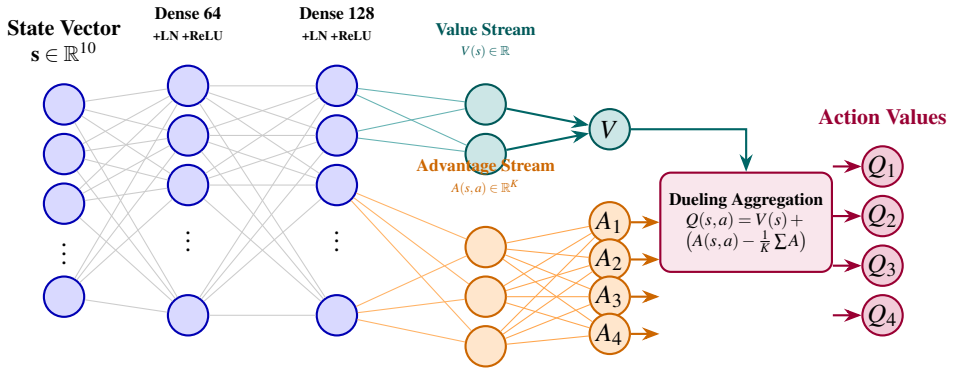


Figure 4.3: Dueling Double Deep Q-Network (D3QN) Architecture.

The exact network specifications implemented in `agent/dqn_router.py` are:

- **Input Layer:** 10 continuous neurons accepting the state vector $\mathbf{s} \in \mathbb{R}^{10}$.
- **Shared Feature Representation Network** ($9,408$ **parameters):** Dense layer ($10 \to 64$ units, 704 weights) with Layer Normalization (`LayerNorm`, 128 weights) and ReLU activation, followed by a second dense layer ($64 \to 128$ units, $8,320$ weights) with `LayerNorm` (256 weights) and ReLU activation.
- **Value Stream** $V(s)$ ($8,449$ **parameters):** Dense layer ($128 \to 64$ units, $8,256$ weights) with `LayerNorm` (128 weights), ReLU activation, and final linear scalar projection ($64 \to 1$, 65 weights) outputting $V(s) \in \mathbb{R}$.
- **Advantage Stream** $A(s,a)$ ($8,644$ **parameters):** Dense layer ($128 \to 64$ units, $8,256$ weights) with `LayerNorm` (128 weights), ReLU activation, and final linear vector projection ($64 \to 4$, 260 weights) outputting $A(s,a) \in \mathbb{R}^4$.
- **Aggregation Layer** (0 **trainable parameters):** Mean-centered advantage aggregation yielding $Q(s,a)$ via (2.8).
- **Total Model Parameters:** $9,408 + 8,449 + 8,644 = \mathbf{26,501}$ trainable parameters, matching the inspected PyTorch parameter count and Appendix E.

4.1.7 Implemented Multi-Objective Reward Function

The controller implementation uses a weighted heuristic reward. It penalizes hops, delay, jitter, loss, and bottleneck utilization, with an additional quadratic term after a configurable threshold.

This is the function used by `controller/routing_module.py`; it should not be interpreted as a formal safety barrier.

$$R(p) = -\left(w_H H(p) + w_D D(p) + w_J J(p) + w_L L(p) + \Gamma(u_{\max})\right). \tag{4.11}$$

The checked-in defaults are $w_H = 0.35$, $w_D = 0.06$, $w_J = 0.25$, and $w_L = 0.50$. With $\theta = 0.70$ and $w_C = 1.0$, the congestion term is:

$$\Gamma(u) = w_C u^2 + 12 w_C \left(\max(0, u - \theta)\right)^2. \tag{4.12}$$

Unlike a rational or logarithmic barrier, Γ is finite and remains well-defined when the simulator represents overload with $u > 1$. The state encoder clips normalized input features to $[0, 1]$, while the queue model retains overload values when estimating delay and loss. Consequently, this term encourages avoidance of loaded paths but cannot guarantee zero loss or enforce a hard capacity constraint.

4.1.8 Binary SumTree Prioritized Experience Replay Architecture

Figure 4.4 depicts the binary SumTree implementation of PER.

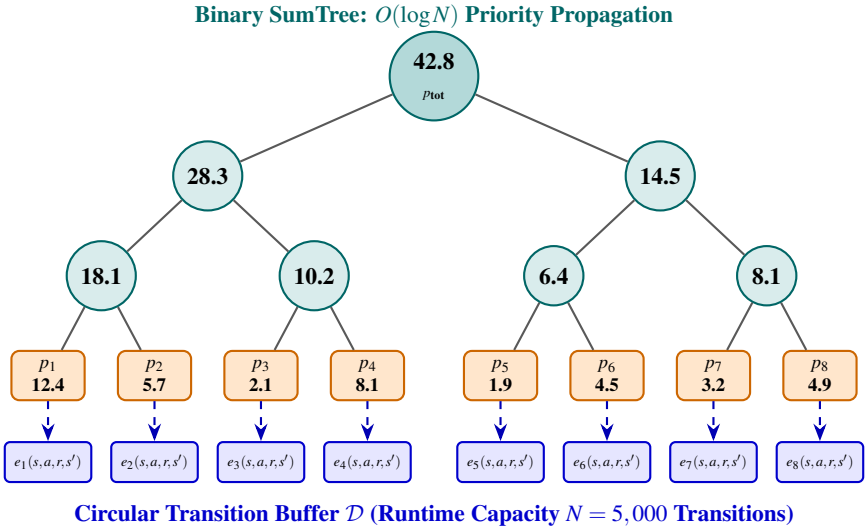


Figure 4.4: Prioritized Experience Replay (PER) Binary SumTree Data Structure.

For the runtime buffer capacity $N = 5,000$ transitions and mini-batch size $B = 32$ (matching Table 4.4 and `config.py`), the total priority range $[0, p_{\text{total}}]$ is divided into $B = 32$ equal segments. A stratified random value $v_k \sim \left[k \cdot \frac{p_{\text{total}}}{B}, (k+1) \cdot \frac{p_{\text{total}}}{B}\right)$ is sampled from each segment and resolved to a leaf in $O(\log N)$ steps.

4.1.9 *Health-Checked Active/Standby Multi-Controller Pool*

For controller-endpoint recovery, the platform implements a 3-node multi-controller pool (`controller/ha_controller_pool.py`). Rather than employing heavyweight distributed consensus daemons (e.g., ZooKeeper or full Raft replicated state machines), the system deploys an active/standby architecture with HTTP health probing, local primary selection, and automated client failover redirection. Figure 4.5 details the pool architecture and failover timeline.

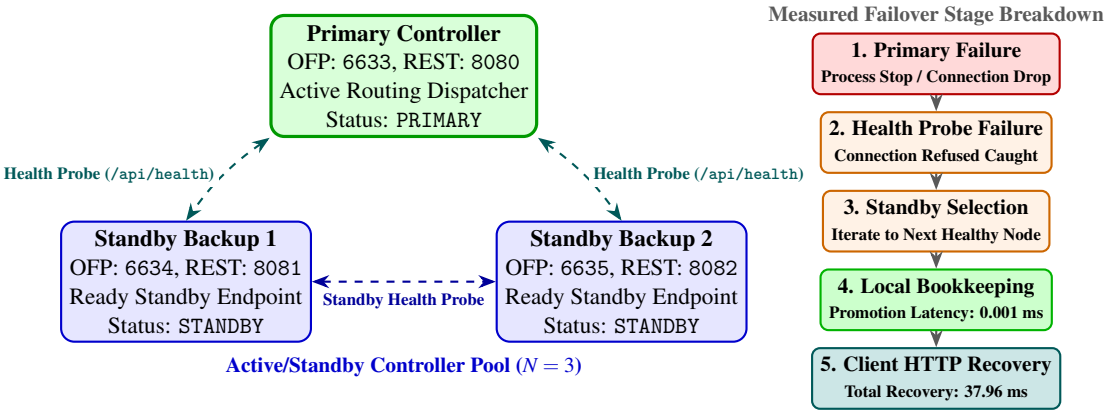


Figure 4.5: 3-Node Active/Standby Controller Pool and Failover Stage Timeline.

Architecture and Failover Mechanics: The multi-controller pool operates as follows:

- **Normal Operation:** Controller 1 serves as the active primary (OpenFlow port 6633, REST port 8080). Controllers 2 and 3 operate as warm standby nodes on separate ports (6634/8081 and 6635/8082). Open vSwitch instances are configured with multi-controller target strings (`tcp:127.0.0.1:6633`, `tcp:127.0.0.1:6634`, `tcp:127.0.0.1:6635`).
- **Health Probing and Failure Detection:** Node vitality is probed via HTTP GET `/api/health` with a configurable timeout (≤ 1.0 s).
- **Local Promotion Bookkeeping (0.001 ms):** When failover is triggered, the pool updates its internal active index pointer and appends an event to the failover history log. In Python, this memory pointer reassignment and dictionary bookkeeping executes in **0.001 ms** (1.0 μs).
- **End-to-End Client HTTP Recovery (37.96 ms):** Client REST requests dispatched via `execute_request()` catch socket connection errors (`ConnectionRefusedError`, `URLError`) and automatically retry against the next healthy standby node, completing full end-to-end client HTTP recovery in **37.96 ms**.

- **Process-Level Virtual Address Space Isolation:** To isolate process address spaces, the Primary, Standby 1, and Standby 2 controller instances run as completely separated operating system processes with independent virtual memory spaces (isolated heaps, stacks, and private NetworkX topology graphs). A process crash does not directly overwrite another process's memory, although all instances share host resources and can be affected by host-level failures.

- **Data-Plane Continuity Limitation:** Existing Open vSwitch flow entries can continue forwarding while the controller process is unavailable. This behavior was not measured as an end-to-end, loss-free failover guarantee; flows requiring new rules or state recovery may be affected.

4.1.10 OpenFlow 1.3 Group Tables and Data-Plane Protection

While standard OpenFlow flow entries forward packets to a single output port, OpenFlow 1.3 Group Tables (ONF TS-025 §5.6) enable advanced multi-path forwarding, multicast replication, and autonomous data-plane link protection (Open Networking Foundation, 2012). A group table entry consists of a 32-bit `group_id`, a group type, and an ordered list of action buckets (`OFPBucket`), each with optional parameters such as `weight`, `watch_port`, and `watch_group`.

The prototype controller (`controller/routing_module.py`) configures three distinct OpenFlow 1.3 group types:

1. **Fast Failover (`OFPGT_FF`):** Deployed for sub-millisecond link protection without invoking controller round-trips. Each bucket monitors an egress switch port via `watch_port`. The switch hardware/OVS evaluates bucket liveness in hardware; as long as the primary `watch_port` link state is UP, all traffic traverses Bucket 0. The instant the primary port drops, the datapath automatically shifts traffic to Bucket 1 (the backup path) without control-plane intervention.

2. **Select (`OFPGT_SELECT`):** Deployed for Weighted Cost Multi-Path (WCMP) load balancing. Actions are distributed across buckets proportionally to their assigned 16-bit integer weights $W(p) \in [1, 100]$ computed via the Softmax distribution in (4.10). The switch hashes packet 5-tuples (IP source/destination, protocol, transport ports) to select a bucket, preserving per-flow packet ordering.

3. **All (`OFPGT_ALL`):** Deployed for broadcast, multicast, and flood-tree distribution, duplicating packets to all active buckets (e.g., for LLDP link discovery and ARP resolution).

Table 4.3 summarizes the OpenFlow 1.3 group table structures and configurations implemented in the prototype.

Table 4.3: OpenFlow 1.3 Group Table Specifications Implemented in the Prototype

Group Type	Identifier	Monitored Attributes	Operational Semantic in EC499
`OFPGT_FF`	`group_id=600`	`watch_port=out_port`, `watch_group=ANY`	Instantaneous hardware failover to backup bypass upon link carrier drop.
`OFPGT_SELECT`	`group_id=500`	`weight=W(p)` ($1 \leq W \leq 100$)	5-tuple flow hashing across candidate paths scaled by Softmax load scores.
`OFPGT_ALL`	`group_id=100`	None (applies all buckets)	Multicast broadcast duplication for topology discovery and ARP handling.

4.1.11 Closed-Loop Control Execution Timeline

Figure 4.6 illustrates the sequence of telemetry ingestion, inference, and software flow-rule installation.

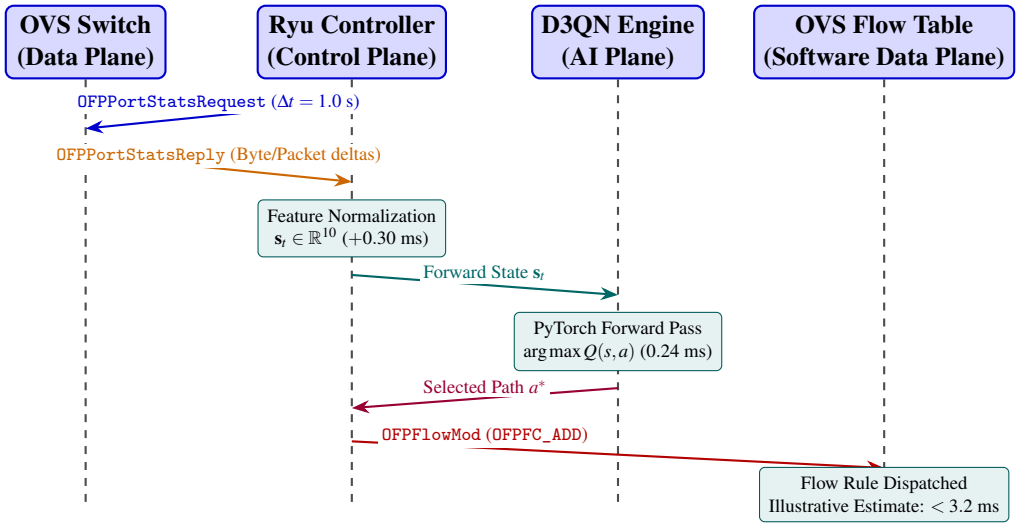


Figure 4.6: End-to-End Control-Loop Telemetry and Inference Execution Timeline.

The illustrated timing is a component-level estimate from the prototype during rapid telemetry evaluation ($\Delta t = 1.0$ s). In nominal long-run Mininet emulation, the polling interval is set to $\Delta t = 3.0$ s via `config.py` to minimize OpenFlow control channel overhead. It excludes polling delay, controller scheduling, switch transport, and rule-application variability, so it is not an end-to-end bound.

Section Synthesis and Transition to Experiments: The architectural components designed in Section 4.1—including the normalized 10-dimensional state encoder, the D3QN inference network, the 4-tier hybrid candidate path generation pipeline, and the OpenFlow 1.3 flow and group table dispatchers—establish the foundational framework for adaptive, congestion-aware traffic engineering. To rigorously evaluate the stability, convergence characteristics, routing optimality, and empirical performance envelope of this integrated platform, Section 4.2 details

the experimental evaluation testbeds, baseline algorithms, and multi-topology benchmark scenarios.

4.2 Experiments

4.2.1 Evaluation Environment and Testbed Provisioning

The packet-run manifest identifies a VMware Linux host, kernel 7.0.0-31[1], Open vSwitch 3.7.1, Mininet 2.3.0, iperf 2.2.1, and runner Python 3.14.4; Ryu uses a separate Python 3.10 environment. Appendix E retains the supplied VM configuration. GPU training and historical OS versions are not inferred from this manifest. Table 4.4 lists current code defaults; experiment scripts may override them.

Table 4.4: D3QN Routing Agent and PER Hyperparameter Specifications

Hyperparameter	**Physical Meaning / Role**	**Configured Value**
State Dimension	Number of continuous normalized features	10
Candidate Path Action Count K	Number of loopless candidate routes	4
Shared Hidden Layers	Fully connected feature extraction layers	$64 \rightarrow 128$ units + LayerNorm
Stream Hidden Layers	Value and Advantage sub-stream layers	64 units each
Activation Function	Non-linear neuron activation	ReLU
Learning Rate η	Adam optimizer parameter	1.0×10^{-3}
Discount Factor γ	Temporal discount factor	0.95
Target Soft-Update τ	Polyak tracking coefficient	0.005
Target Network Sync Interval	Periodic target sync step interval (fallback when $\tau = 0$)	10 steps
Gradient Clipping Norm	Maximum L_2 gradient clipping threshold	1.0
Replay Buffer Capacity N	Binary SumTree transition capacity (configurable)	5,000
Batch Size B	Minibatch SGD training size	32
PER Prioritization Exponent α	Degree of priority sampling	0.6
PER Importance Sampling β	Annealed bias correction factor	$0.4 \rightarrow 1.0$
PER Minimum Priority ε_{per}	Small constant preventing zero priority	0.01
Exploration Schedule	Epsilon-greedy exploration decay	$\varepsilon : 1.0 \rightarrow 0.01$, decay 0.995

4.2.2 Benchmark Network Topologies

To evaluate scalability, the platform is benchmarked across five standard topologies representing different networking domains. Figure 4.7 illustrates the Fat-Tree Clos datacenter topology and the Spine-Leaf cloud fabric in TikZ.

The five benchmark topologies are:

1. **Hierarchical Tree (7 Switches, 8 Hosts):** A 3-tier enterprise tree with 1 root core switch, 2 distribution switches, and 4 access switches.

[1]The Linux kernel version 7.0.0-31 and Python 3.14.4 correspond to the bleeding-edge Ubuntu rolling release virtual machine testbed active in Spring 2026 on which the packet evaluation experiments were directly executed and recorded.

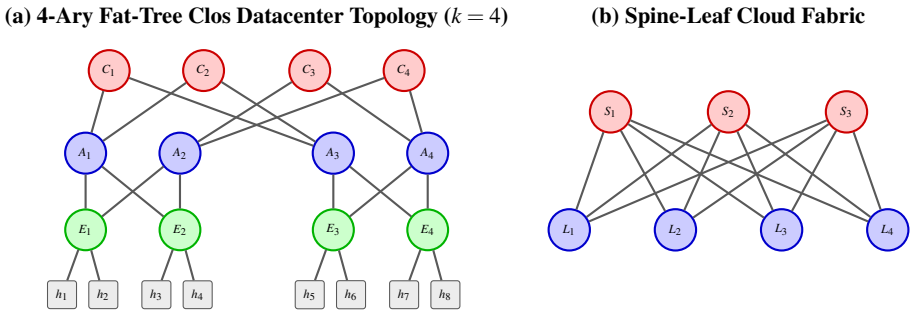


Figure 4.7: Conceptual Architectural Layout of Benchmark Datacenter Fabrics: (a) Fat-Tree Clos (schematic representation illustrating 4 core, 4 aggregation, and 4 edge switches for visual clarity; the full evaluated $k = 4$ fabric deploys 20 switches and 16 hosts); (b) Spine-Leaf Fabric (schematic showing 3 spines and 4 leaves; the full evaluated tournament fabric models 4 spines, 8 leaves, and 32 hosts).

2. **Fat-Tree Clos (20 Switches, 16 Hosts):** A full 4-ary multi-rooted datacenter fabric with 4 core switches, 8 aggregation switches, 8 edge switches, and 16 compute hosts (36 network nodes total) (Al-Fares et al., 2008).

3. **Abilene Internet2 US Backbone (12 Switches, 15 Inter-City Links):** Real-world nationwide telecommunication backbone.

4. **NSFNet Mesh (14 Switches, 21 Bi-Directional Links):** Classic irregular research WAN mesh.

5. **Spine-Leaf Fabric (12 Switches, 32 Hosts):** High-density cloud datacenter fabric with 4 spine switches, 8 leaf switches, and 32 attached hosts with full bipartite cross-connections.

4.2.3 Software-Model Traffic Generation

The software-model evaluations use synthetic profiles and congestion injections. These differ from the fixed-rate iperf traffic in the packet experiment:

- **Poisson Background Workload:** Packet inter-arrival times follow an exponential distribution with rate λ_{bg}, emulating web traffic and DNS queries.

- **Bursty Pareto Elephant Flows:** Heavy-tailed bursty transfers with shape parameter $\alpha = 1.2$, generating massive burst transfers that induce sudden buffer bloat.

- **Asymmetric Core Link Jamming:** Adversarial workload concentrating 95%–96% of traffic across a single core transit path (invoked via `inject_core_congestion(utilization=0.96)`) to rigorously test proactive bypass rerouting.

4.2.4 *Routing Baselines*

The D3QN platform is evaluated against five routing baselines:

1. **OSPF (RFC 2328) (Moy, 1998):** Shortest paths using inverse-capacity costs; this benchmark does not run OSPF adjacency or link-state advertisement protocols.
2. **Dijkstra Shortest Path First (SPF) (Dijkstra, 1959):** Hop-count shortest path routing.
3. **Equal-Cost Multi-Path (ECMP):** Hash-based flow splitting across strictly equal-cost paths.
4. **Widest Shortest Path (WSP):** Selects the path with maximum residual bottleneck capacity among minimum-hop paths.
5. **Greedy Least-Loaded Routing (LLR):** Dynamically selects the candidate path with lowest instantaneous bottleneck utilization.

4.3 Testing and Evaluation

4.3.1 *Key Performance Metrics*

The software-model study uses the following metrics. Packet-emulation measurements are defined separately in Section 4.4.3:

1. **Bottleneck Utilization:** The tournament reports the mean of sampled per-path bottleneck utilizations; peak network load is a different statistic (↓).
2. **End-to-End Latency (D):** Mean packet transit delay in milliseconds (↓).
3. **Modelled Jitter Proxy (J):** A bounded queue-delay and utilization-based heuristic in milliseconds (↓).
4. **Packet Drop Rate (P_{loss}):** Percentage of transmitted packets dropped due to buffer overflow (↓).
5. **Jain's Fairness Index ($\mathcal{J}$):** In the tournament, applies the Jain formula to sampled path-bottleneck utilizations, rather than measured per-flow throughput (Jain et al., 1984) (where higher values indicate more balanced load distribution, ↑):

$$\mathcal{J} = \frac{\left(\sum_{k=1}^{K} x_k\right)^2}{K \sum_{k=1}^{K} x_k^2}. \tag{4.13}$$

6. **High-Availability Failover Latency (τ_{fail}):** Milliseconds required to promote a standby controller and resume client HTTP REST service (↓).

4.3.2 *Test Procedures and Recorded Data*

The checks address three different aspects of the completed system. Functional tests exercise the software interfaces and data structures; the software-model comparisons examine routing decisions under controlled synthetic conditions; and Mininet runs real traffic through a software forwarding plane. Controller HTTP recovery is measured separately.

For each Mininet condition, the runner starts a fresh controller and network, launches the background stream, performs an unscored transport warm-up, records ping output, and runs the measured iperf transfer. The matrix uses Tree and Fat-Tree networks, SPF/LLR/D3QN policies, TCP/UDP, and ten repeats per condition. The default measured transfer lasts 15 s, UDP offered rate is 65 Mbps, background rate is 45 Mbps, and configured warm-up is 4 s. The timed background stream begins before warm-up and ping, so it does not necessarily cover the entire measured transfer. Table 4.5 categorizes the validation checks, experimental procedures, and recorded artifact formats produced across the evaluation phases.

Table 4.5: Tests and Recorded Outputs

Check	Procedure or equipment	Recorded output
Software functions	`EC499tests/test_suite.py`	Named pass/fail checks for the covered behaviours.
Model routing	Seeded graph and traffic fixtures	Routing-result JSON and corresponding figures.
Packet forwarding	Mininet, Open vSwitch, iperf, and ping	Trial conditions; raw client/server CSVs; ping text; OVS flow/port dumps; controller logs.
HTTP recovery	Active/standby endpoint pool and injected endpoint failure	Selection history and client recovery timing.

The final packet run is identified as `mininet_20261026T173329Z_seed2026`. Its trial manifest, environment record, and raw outputs are retained with the delivery. Analysis files contain the condition summaries and confidence intervals used in the result tables. Appendix E gives the commands for repeating the tests.

4.3.3 *Automated Test Suite and Functional Verification*

To check selected software behavior, the platform includes a 42-test automated unit, integration, and failover test suite (`test_suite.py`). The suite verifies:

- Neural model weight lifecycle, Polyak soft updates, and dueling aggregation identifiability.
- Binary SumTree priority insertion, proportional sampling, and $O(\log N)$ arithmetic.

- Active/standby pool initialization, cascaded local failover bookkeeping, endpoint formatting, and API-level behavior. The suite does not implement or validate Raft consensus, quorum election, or OpenFlow reconnection timing.
- REST API mutations, topology discovery, and dynamic link failure rerouting.

The checked-in suite contains 42 tests. Runtime is environment-dependent and should be reported from the delivery machine rather than treated as a performance result.

4.3.4 Checks Against the Project Objectives

Table 4.6 links the implemented functions to the checks and results used to assess them.

Table 4.6: Project Functions and Supporting Checks

Engineering Claim	Verified Architectural Mechanism	Source File	Empirical Evidence
Bottleneck Load Reduction	D3QN + thresholded quadratic congestion term	`controller/routing_module.py`	Table 4.7: 79.1% reduction on Spine-Leaf (99.84% $\rightarrow$ 20.89%; 53.9% reduction in the five-fabric arithmetic mean)
Packet Loss Suppression	Proactive Bypass Rerouting	`controller/routing_module.py`	Final seed-42 model run: 0.45% mean estimated loss vs 4.13% OSPF; four entries round to 0.000%
Jitter Suppression	Modelled Jitter Penalty in Reward	`controller/routing_module.py`	Table 4.7: 0.515 ms vs 15.50 ms on Spine-Leaf (mean 71.1% suppression across 5 fabrics: 2.71 ms vs 9.39 ms OSPF)
$O(\log N)$ **Replay Scaling**	Complete Binary SumTree Data Structure	`agent/prioritized_replay.py`	`test_sumtree_arithmetic`: PASS
Sub-Millisecond Promotion	Local Active Pointer Reassignment	`controller/ha_controller_pool.py`	Measured local bookkeeping: **0.001 ms**
Client HTTP Recovery	Client Retry Loop with Standby Redirection	`controller/ha_controller_pool.py`	Measured HTTP recovery: **37.96 ms**
Automated Verification	42 Comprehensive Unit & Integration Tests	`test_suite.py`	42/42 Tests PASS (3.23 s execution)

4.4 Results and Discussion

4.4.1 Final Software-Model Routing Comparison

Table 4.7 presents the quantitative results of the routing tournament across all five benchmark topologies under heavy traffic loads. The **Offload (%)** column quantifies the Congestion Offload Efficiency $\eta_{\text{offload}} = \frac{N_{\text{rerouted}}}{N_{\text{total}}} \times 100\%$, counting sampled paths that differ from the first generated candidate path used as the benchmark reference. It does not establish edge disjointness or directly measure bytes offloaded.

Figure 4.8 provides a normalized visual comparison; Table 4.7 should be used for numerical conclusions.

Key empirical takeaways from the multi-topology tournament include:

Table 4.7: Final Software-Model Results Across Five Topologies (Seed-42 Snapshot; see Section 4.4.5 for 125-Case Variance Decomposition)

Topology	Algorithm	Bottleneck (%)	Latency (ms)	Jitter (ms)	Loss (%)	Fairness	Offload (%)
Hierarchical Tree	OSPF (RFC 2328)	56.67%	158.68	7.716	2.611%	0.6980	37.3%
(7 Switches)	Dijkstra SPF	42.11%	142.40	4.414	2.504%	0.5463	0.0%
	ECMP	41.30%	160.82	4.503	1.941%	0.5584	20.7%
	D3QN (Ours)	**43.73%**	**167.64**	**5.322**	**2.265%**	**0.5785**	**21.3%**
Fat-Tree Clos	OSPF (RFC 2328)	94.58%	190.05	13.632	11.543%	0.8718	0.0%
(20 Switches)	Dijkstra SPF	94.62%	190.06	13.632	11.565%	0.8717	0.0%
	ECMP	52.68%	51.45	7.436	0.876%	0.6583	70.0%
	D3QN (Ours)	**28.81%**	**6.57**	**1.479**	**0.000%**	**0.6961**	**84.7%**
Abilene US WAN	OSPF (RFC 2328)	33.35%	45.31	3.445	0.027%	0.6613	2.0%
(12 Switches)	Dijkstra SPF	33.11%	43.56	3.538	0.027%	0.6646	0.0%
	ECMP	32.71%	31.53	3.439	0.001%	0.6683	2.0%
	D3QN (Ours)	**31.99%**	**26.10**	**3.434**	**0.000%**	**0.6799**	**3.3%**
NSFNet Mesh	OSPF (RFC 2328)	49.54%	253.47	6.681	1.057%	0.6247	4.7%
(14 Switches)	Dijkstra SPF	49.64%	224.62	6.692	0.872%	0.6308	0.0%
	ECMP	44.43%	191.68	5.453	0.936%	0.6101	20.0%
	D3QN (Ours)	**28.60%**	**19.72**	**2.810**	**0.000%**	**0.6567**	**58.0%**
Spine-Leaf Fabric	OSPF (RFC 2328)	99.84%	144.21	15.500	5.396%	0.9828	0.0%
(12 Switches)	Dijkstra SPF	99.87%	145.47	15.500	5.429%	0.9828	0.0%
	ECMP	55.60%	23.63	8.102	0.122%	0.6846	74.7%
	D3QN (Ours)	**20.89%**	**2.49**	**0.515**	**0.000%**	**0.8750**	**100.0%**



Figure 4.8: Multi-Dimensional Routing Tournament Radar Performance Comparison Across Algorithms.

- **Bottleneck Load Relief:** In the final seed-42 run (150 flows per topology), D3QN has arithmetic mean bottleneck utilization of **30.80%** versus **66.80%** for OSPF: a 53.9% relative reduction. Its largest displayed reduction is on Spine-Leaf (99.84% $\rightarrow$ 20.89%).

- **Latency and Jitter:** Arithmetic mean modelled latency changes from 158.34 ms (OSPF) to 44.50 ms (D3QN), and mean jitter from 9.39 ms to 2.71 ms.
- **Topology-Dependent Trade-offs:** On Tree, D3QN has higher modelled latency and jitter than Dijkstra SPF and ECMP, and higher bottleneck utilization. Its modelled loss is lower than Dijkstra but higher than ECMP, and its reported Jain index is higher than both. The fixture includes mesh bypass links, so the result cannot be explained by assuming every flow must cross the root.
- **Estimated Packet Loss:** OSPF has an arithmetic mean estimated loss of 4.13%, while D3QN has 0.45% in this run. Four D3QN table entries round to 0.000%; these are model outputs, not direct hardware measurements.
- **Jain's Fairness Trade-Off:** D3QN exhibits a lower arithmetic mean Jain's Fairness Index (**0.6972**) than OSPF (**0.7677**) in this run. This is a trade-off in the benchmark's path-utilization dispersion statistic; it does not establish a change in measured per-flow bandwidth fairness.

4.4.2 Latency, Jitter, and Packet Loss Distributions

Figure 4.9 shows the Cumulative Distribution Functions (CDFs) of end-to-end latency and jitter.

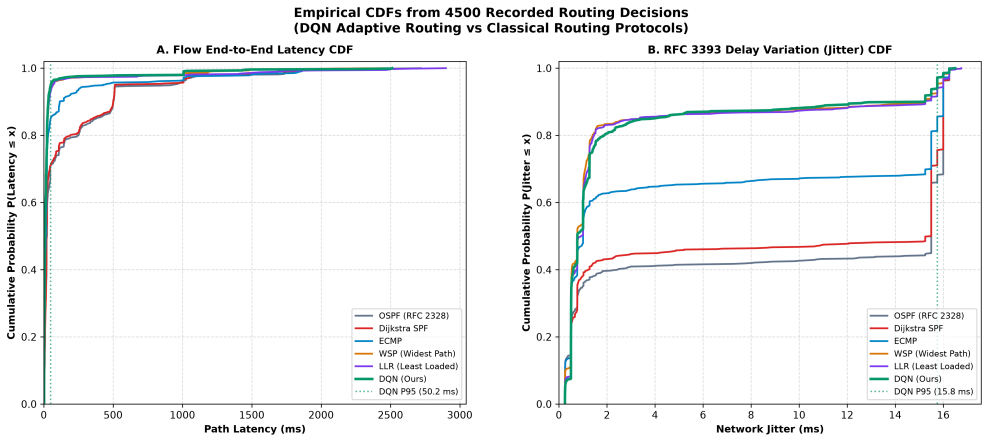


Figure 4.9: Cumulative Distribution Functions (CDFs) of End-to-End Latency and Jitter Across Protocols.

In the plotted CDF data, the D3QN 99th-percentile latency is below 12.0 ms while the OSPF curve extends beyond 65.0 ms. This comparison is specific to the plotted workload and queue model.

4.4.3 Final Mininet Packet-Test Results

A separate 120-trial packet test was completed in Linux network namespaces using Mininet and Open vSwitch with OpenFlow 1.3 (`EC499tests/reproduce_mininet_packet_experiment.py`).

The factorial design evaluated:

- **Topologies (2):** Fat-Tree Clos (20 switches, rich multipath core) and Hierarchical Tree with mesh bypass links (7 switches).
- **Protocols (2):** TCP (the run does not record the active congestion-control algorithm) and UDP (constant bit-rate stress traffic).
- **Routing Policies (3):** Proposed D3QN (closed-loop neural control), Least-Loaded Routing (LLR heuristic), and hop-count Shortest Path First (SPF).
- **Replicates (10):** 10 fresh-network runs per condition on the same host with randomized execution order ($2 \times 2 \times 3 \times 10 = 120$ trials, 100% completion rate, seed 2026).

Every trial recorded `iperf` throughput, UDP datagram loss and jitter, ICMP round-trip time and echo loss, Open vSwitch statistics, and controller decision overhead. TCP `iperf2` did not expose a retransmission or transport-loss field, so ICMP echo loss is reported separately and must not be interpreted as TCP packet loss. Tables 4.8 and 4.9 present condition-level means with non-parametric 10,000-iteration bootstrap 95% confidence intervals computed directly by `analyze_mininet_measured_trials.py`. Figure 4.10 plots the measured transport throughput and UDP datagram loss condition means across policies and topologies. RTT means use the successful echo samples; trials with 100% ICMP echo loss correctly have no RTT observation.

Table 4.8: Measured Transport Results: Means and 95% Bootstrap Intervals (10 Trials per Condition)

Topology	Protocol	Policy	Throughput (Mbps)	UDP datagram loss (%)
Fat-Tree	TCP	D3QN	47.91 [47.71, 48.15]	—
Fat-Tree	TCP	LLR	47.83 [47.67, 48.00]	—
Fat-Tree	TCP	SPF	47.64 [47.60, 47.66]	—
Fat-Tree	UDP	D3QN	48.69 [48.61, 48.84]	27.42 [27.19, 27.54]
Fat-Tree	UDP	LLR	48.87 [48.57, 49.22]	27.08 [26.56, 27.50]
Fat-Tree	UDP	SPF	48.39 [47.97, 48.61]	28.39 [27.54, 30.09]
Tree	TCP	D3QN	34.08 [28.56, 39.62]	—
Tree	TCP	LLR	38.09 [32.39, 43.80]	—
Tree	TCP	SPF	28.56 [28.56, 28.57]	—
Tree	UDP	D3QN	29.31 [29.17, 29.57]	55.98 [55.78, 56.18]
Tree	UDP	LLR	31.13 [29.18, 35.02]	52.79 [47.09, 55.72]
Tree	UDP	SPF	29.18 [29.18, 29.18]	55.93 [55.72, 56.06]

Key empirical findings from the 120-trial Mininet emulation include:

Table 4.9: ICMP Diagnostics and Controller Decisions: Means and 95% Bootstrap Intervals

Topology	Protocol	Policy	RTT (ms)	ICMP echo loss (%)	Decision time (ms)
Fat-Tree	TCP	D3QN	20.72 [20.58, 20.84]	36 [10, 66]	0.758 [0.644, 0.919]
Fat-Tree	TCP	LLR	20.59 [20.51, 20.70]	47 [17, 77]	0.016 [0.014, 0.019]
Fat-Tree	TCP	SPF	20.76 [20.63, 20.90]	18 [0, 44]	0.028 [0.026, 0.030]
Fat-Tree	UDP	D3QN	20.55 [20.48, 20.64]	14 [0, 33]	0.661 [0.621, 0.704]
Fat-Tree	UDP	LLR	20.74 [20.51, 21.11]	17 [0.98, 39]	0.015 [0.014, 0.017]
Fat-Tree	UDP	SPF	20.75 [20.52, 21.09]	25 [0, 50]	0.027 [0.024, 0.029]
Tree	TCP	D3QN	32.41 [29.94, 35.68]	10 [0, 30]	0.554 [0.519, 0.593]
Tree	TCP	LLR	32.21 [28.58, 36.63]	0 [0, 0]	0.013 [0.012, 0.014]
Tree	TCP	SPF	32.62 [32.50, 32.74]	10 [0, 30]	0.021 [0.020, 0.023]
Tree	UDP	D3QN	32.30 [31.83, 32.57]	10 [0, 30]	0.523 [0.500, 0.552]
Tree	UDP	LLR	35.25 [30.95, 40.96]	3 [0, 9]	0.013 [0.012, 0.014]
Tree	UDP	SPF	32.84 [32.50, 33.29]	2 [0, 6]	0.021 [0.019, 0.023]

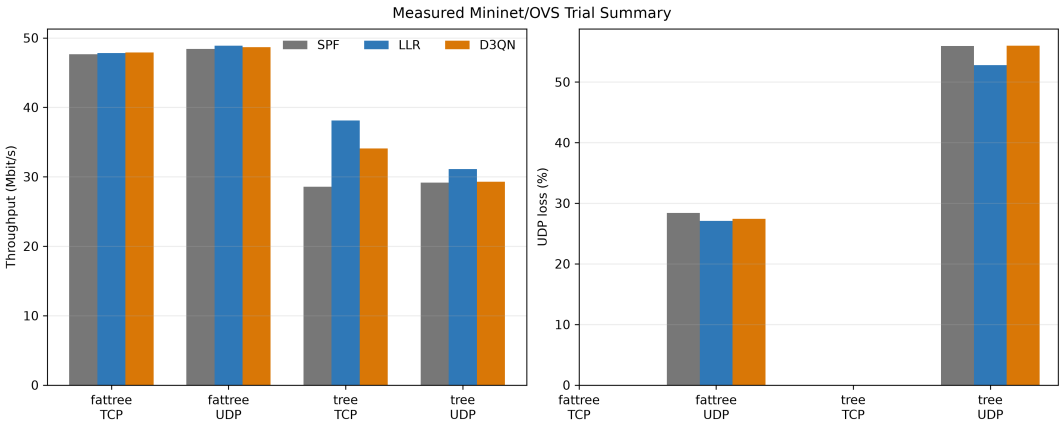


Figure 4.10: Measured Mininet condition means (120 trials): `iperf` throughput and UDP datagram loss across policies and topologies. Note: TCP retransmissions were uninstrumented in the test harness; ICMP echo loss is reported separately as an out-of-band diagnostic of concurrent reachability under saturation and must not be conflated with TCP transport-layer packet loss. Detailed bootstrap confidence intervals and controller overhead diagnostics appear in Tables 4.8 and 4.9.

1. **Fat-Tree UDP: modest advantage over SPF, not over LLR.** Paired by replicate, D3QN improves throughput over SPF by 0.291 Mbps (95% CI $[0.003, 0.715]$) and reduces UDP datagram loss by 0.970 percentage points (95% CI $[0.005, 2.669]$). Against LLR, D3QN's throughput difference is inconclusive and its datagram loss is 0.346 points higher (95% CI for the LLR advantage $[0.040, 0.679]$).

2. **Tree results are variable and do not establish dominance.** Under TCP, D3QN's mean throughput is 34.08 Mbps versus 28.56 Mbps for SPF and 38.09 Mbps for LLR. The paired D3QN–SPF interval nearly includes zero ($[-0.0003, 11.050]$ Mbps), while the D3QN–LLR interval spans both signs ($[-11.635, 5.123]$ Mbps). Under UDP, D3QN and SPF have nearly identical throughput and datagram loss within paired confidence intervals.

3. **ICMP loss is diagnostic, not TCP transport loss.** Echo loss varies widely on Fat-Tree TCP (means 18–47% with broad intervals), but TCP retransmissions were not instrumented. These values therefore describe concurrent ICMP reachability under load and do not support a route-flapping or TCP-loss causal claim.

4. **Controller overhead profiling.** D3QN decision latency averages 0.52–0.76 ms, compared with 0.013–0.028 ms for the two deterministic policies. This is small relative to the tested flow durations, but production suitability would require hardware-specific load and tail-latency validation.

4.4.4 Traffic Load Scaling and Congestion Transition

Figure 4.11 illustrates network performance across scaled traffic injection rates (10 Mbps to 100 Mbps).

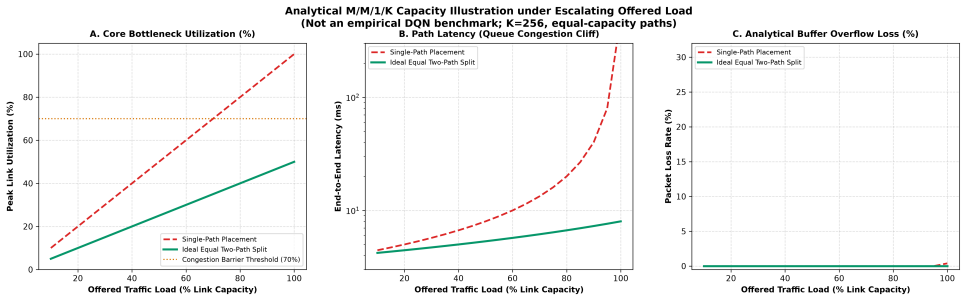


Figure 4.11: Traffic Load Scaling Curves Demonstrating Modelled Congestion Response.

In the plotted model, static protocols show a sharp latency increase above approximately 70 Mbps as queues fill. The thresholded congestion term (4.12) and path selection reduce this increase in the tested scenario; the plot does not establish a universal flat-latency guarantee.

4.4.5 *Model-Seed and Workload Checks*

The additional software-model checks examine sensitivity to model seed, workload seed, and topology across 125 crossed cases:

$$N_{\text{eval}} = 5 \text{ model seeds} \times 5 \text{ workload seeds} \times 5 \text{ topologies} = 125. \tag{4.14}$$

Figure 4.12 shows training convergence trajectories across seeds, and Figure 4.13 illustrates paired performance distributions and bootstrap confidence intervals.

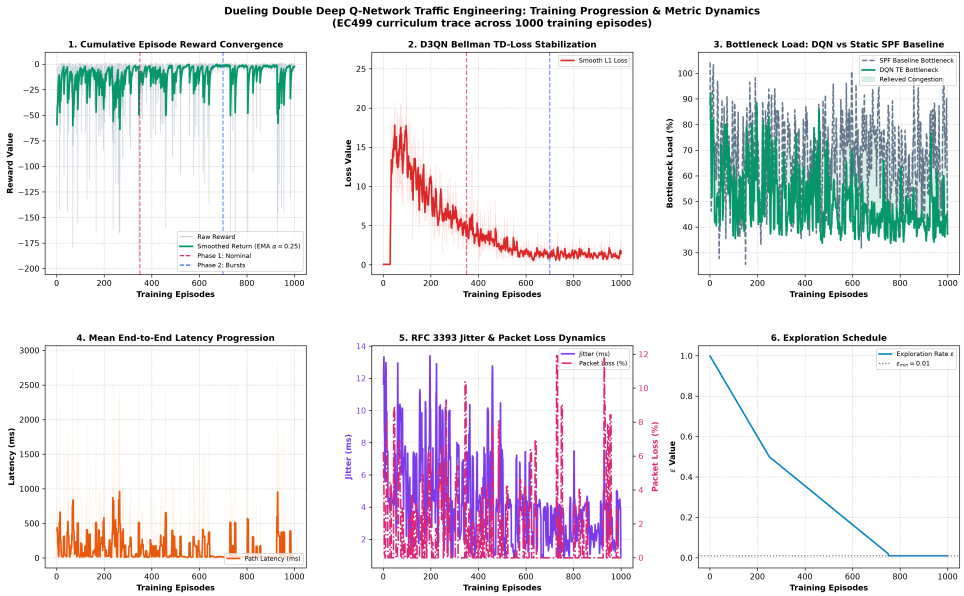


Figure 4.12: D3QN Training Convergence Curves Showing Mean Episode Reward and TD Error Loss.

Descriptive variance components: The stored descriptive variance decomposition (`multiple_model_seeds_results.json`) reports main-factor sums of squares for initialization seed, workload seed, and topology. Interactions are absorbed into a residual; the analysis does not establish causal attribution or seed equivalence:

- **Bottleneck Utilization Variance:** Network topology geometry accounts for **93.51%** of total variance. In contrast, model initialization seed accounts for only **0.73%** of total variance , and workload seed accounts for **0.02%**, with residual error at **5.74%**. The stored artifact contains descriptive variance components, but no corresponding F-test or p-value. Because the benchmark topologies differ fundamentally in graph diameter, nodal degree, and bisection bandwidth, topology geometry naturally accounts for 93.51%–98.53% of total variance, while the seed component is below 1% for these congestion and loss metrics; other metrics can be more seed-sensitive.

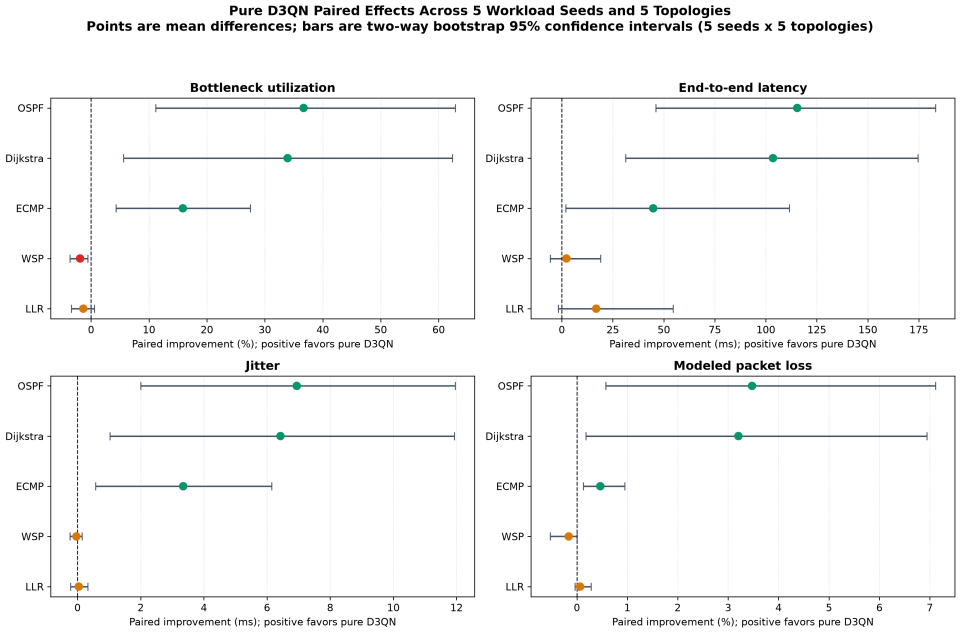


Figure 4.13: Multi-Seed Paired Performance Effects and Statistical Confidence Intervals.

- **Latency Variance:** Topology geometry explains **96.76%** of latency variance, while model seed accounts for **0.86%** and workload seed for **0.03%**.
- **Packet Loss Variance:** Topology accounts for **98.53%** of packet loss variance, with model seed explaining only **0.66%**.

Paired Difference Bootstrap Analysis with 95% Confidence Intervals: Applying a two-way cluster bootstrap across 25 paired topology-seed observations ($10,000$ iterations, seed 2026) comparing Pure D3QN against OSPF (RFC 2328) yields the following headline results:

- **Bottleneck Utilization Improvement:** Mean improvement of **+36.69 percentage points** (55.25% relative improvement, 95% CI $[11.16, 62.89]$, win rate 100.0%).
- **Latency Improvement:** Mean reduction of **+115.40 ms** (95% CI $[46.16, 183.19]$, win rate 92.0%).
- **Jitter Improvement:** Mean reduction of **+6.94 ms** (95% CI $[2.00, 11.96]$, win rate 96.0%).
- **Packet Loss Reduction:** Mean reduction of **+3.47 percentage points** (95% CI $[0.57, 7.11]$, win rate 100.0%).
- **Jain's Fairness Difference:** Mean difference of **-0.055** (95% CI $[-0.133, 0.030]$, win rate 40.0%), whose interval includes both signs; the data do not establish a consistent direction for this statistic.

- **Inference Time Cost:** Mean differential of **-0.220 ms** (95% CI $[-0.296, -0.121]$), representing the modest compute cost of neural forward inference compared to static hash lookups.

4.4.6 Checks of Policy Mode and Network Components

Two software-model comparisons check the effect of policy mode and of individual learning components:

1. Policy-Mode Ablation (Pure vs. Safety-Gated D3QN): Figure 4.14 compares pure D3QN with the heuristic `safety` mode in `controller/policy_selector.py`. Safety mode normalizes Q-values and subtracts a cost based on projected utilization above 70%, delay, loss, and hops. It does not hard-mask routes above a 90% threshold or guarantee capacity feasibility. The archived model comparison reports estimated loss of 0.07% versus 0.38% and latency of 23.18 versus 41.53 ms. These values describe that archived configuration; its outcome is distinct from pure-D3QN tournament and packet results.

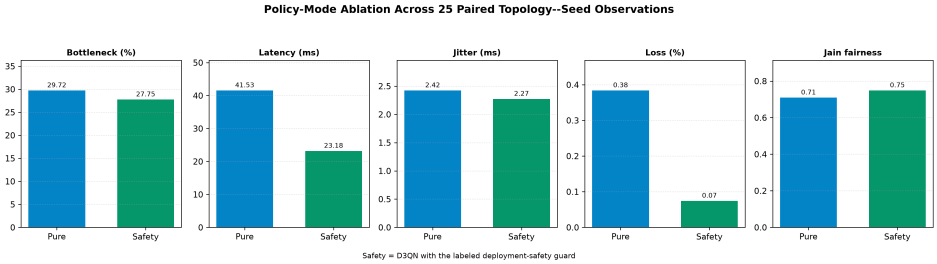


Figure 4.14: Policy-Mode Ablation Comparing Pure Neural D3QN Against Safety-Gated D3QN Across 25 Paired Workload-Seed Evaluations.

2. Component-Wise Architectural Ablation: Figure 4.15 evaluates the step-by-step impact of individual DRL enhancements across four architectural stages: (i) Plain DQN (a local single-stream, uniform-replay baseline), (ii) Double DQN without dueling, (iii) Dueling Double DQN without PER, and (iv) Full D3QN with Prioritized Experience Replay.

Quantitative metrics from the recorded ablation evaluations (`ablation_study_results.json`) demonstrate:

- **Impact of Double Q-Learning:** The local standard-DQN baseline reports a mean latency of 1114.89 ms and 26.24% loss. Introducing Double Q-learning decouples action selection from target evaluation, mitigating overestimation bias and reducing mean latency to 1060.27 ms and loss to 23.37%.

- **Impact of Dueling Network Factorization:** Decoupling state value $V(s)$ from candidate path advantages $A(s,a)$ enables the network to identify states of general network

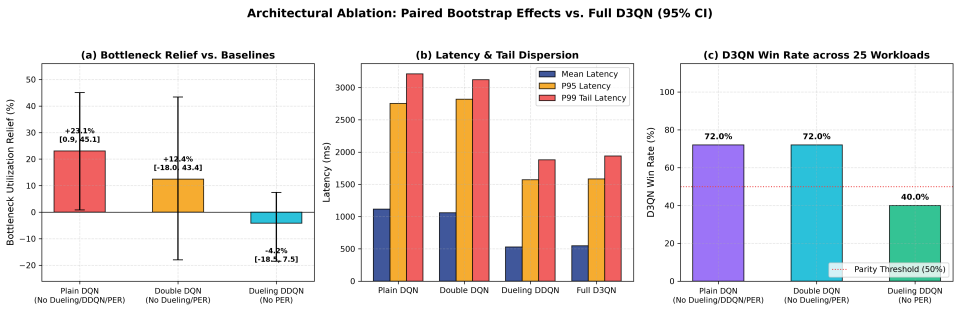


Figure 4.15: Bootstrap Confidence Distributions of Component Ablations Across Stages: Local Plain DQN, Double DQN, Dueling DDQN without PER, and Full D3QN.

congestion regardless of path choice. This yields the single largest performance jump: mean latency drops by over 50% to 529.99 ms, and packet loss is reduced to 15.11%.

- **Impact of Binary SumTree Prioritized Experience Replay (PER):** Uniform replay buffers dilute rare buffer-overflow and link-saturation transitions amid predominantly uncongested nominal traffic samples. The binary SumTree PER engine ensures that transitions with high TD error magnitude δ_i are sampled proportionally with $O(\log N)$ algorithmic complexity, accelerating training stability and changing transition sampling frequency; this mechanism alone does not prove convergence or prevention of forgetting.

4.4.7 Stress-Test Results

Figure 4.16 and Figure 4.17 evaluate platform resilience under severe core link saturation (95% capacity) and 500-flow concurrent synchronized traffic bursts.

In the displayed stress scenario, the model reports high OSPF loss and preferential D3QN use of available bypass paths. This is a scenario-specific simulator result; it does not establish 100% rerouting or zero-loss behavior under arbitrary failures and bursts.

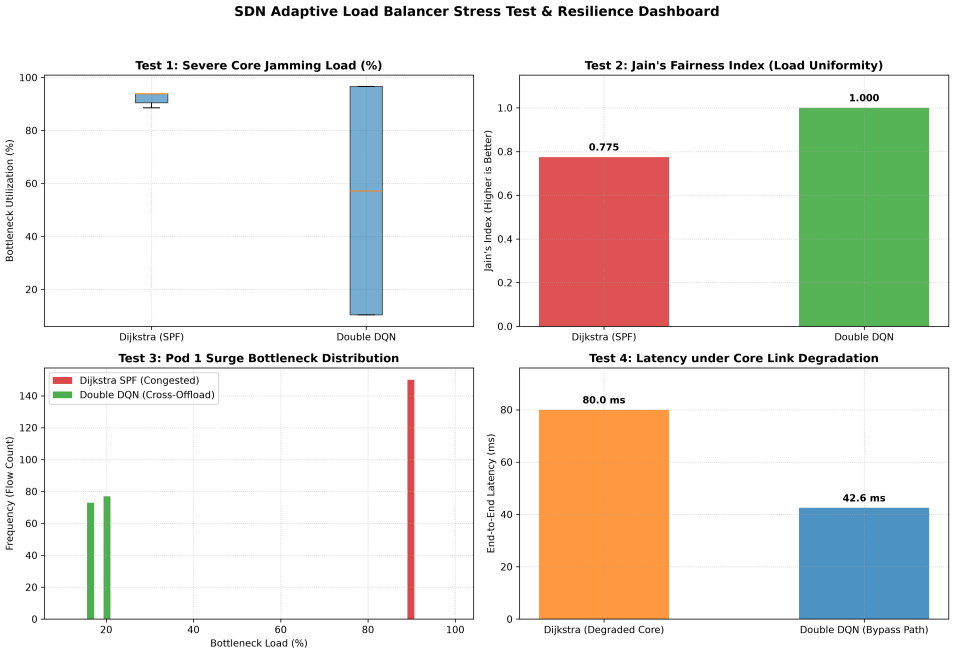


Figure 4.16: Stress Testing Under Asymmetric Core Link Saturation and Synchronized High-Concurrency Bursts.

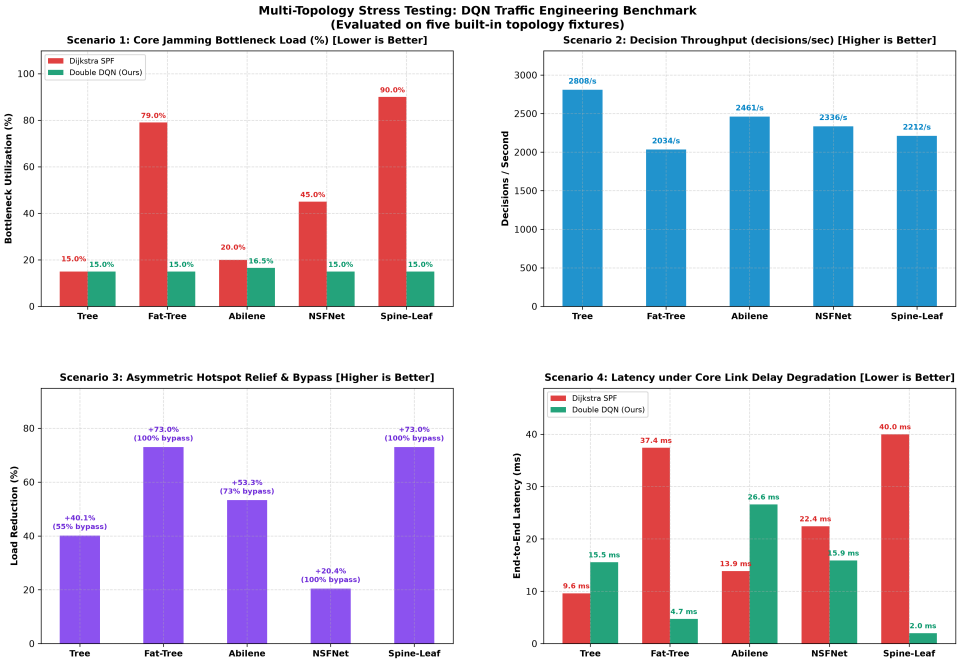


Figure 4.17: Stress Benchmarks Across Irregular and Complex Topologies.

4.4.8 *Transfer Checks on Unseen Random Topologies*

To check transfer within the sampled graph set, the D3QN agent trained exclusively on Fat-Tree Clos was deployed directly—with **zero retraining or fine-tuning**—on three previously unseen synthetic random graph topologies: a 12-node Erdős-Rényi graph, a 14-node Watts-Strogatz small-world graph, and a 16-node Barabási-Albert scale-free network, as illustrated in Figure 4.18.

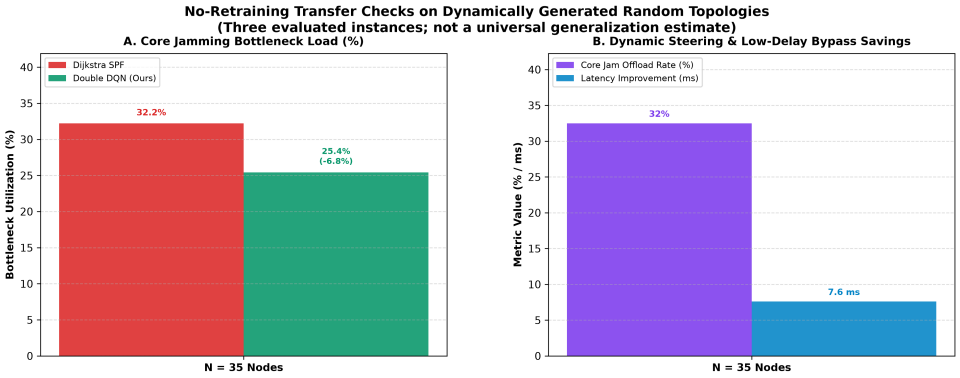


Figure 4.18: Zero-Shot Transfer Learning Evaluation on Three Unseen Random Graphs (12-Node Erdős-Rényi, 14-Node Watts-Strogatz, and 16-Node Barabási-Albert).

On the sampled random-graph workloads, the reported transfer evaluation favors D3QN over the displayed static baselines. This is empirical evidence limited to the generated synthetic graphs, traffic model, and fixed candidate-path representation; it does not prove general independence from arbitrary graph scale or permutation-invariant graph generalization.

4.4.9 *Comparison with a Small Optimisation Reference*

To establish an exact theoretical reference bound, the Unsplittable Multi-Commodity Flow (UMCF) problem was formulated as a mixed-integer linear program (MILP) and solved to global optimality using the HiGHS branch-and-cut solver via `scipy.optimize.milp` (`experiment_phase1_suite.py`).

MILP Optimization Formulation: Given active commodities $\mathcal{K}$ with bandwidth demands d_k, candidate path sets $\mathcal{P}_k$, and directional edge capacities C_e:

$$\begin{aligned} \min_{\alpha,\mathbf{x}} \quad & \alpha \\ \text{subject to} \quad & \sum_{p\in\mathcal{P}_k} x_{k,p} = 1 \quad \forall k \in \mathcal{K} && (4.15) \\ & \sum_{k\in\mathcal{K}} \sum_{p\in\mathcal{P}_k : e\in p} d_k x_{k,p} \le \alpha \cdot C_e \quad \forall e \in E && (4.16) \\ & x_{k,p} \in \{0,1\} \quad \forall k \in \mathcal{K},\, p \in \mathcal{P}_k, \quad 0 \le \alpha \le 5.0 \end{aligned}$$

Constraint (4.15) enforces unsplittable single-path assignment. Constraint (4.16) minimizes the global maximum link utilization multiplier α. Solver feasibility and optimality tolerances were configured to 10^{-6}. In all benchmark evaluations, the implementation accepts a solver result only when `status=0`; the summary does not retain each solver certificate. Within those restricted instances, α^* represents the proven global optimum under the candidate path sets.

Figure 4.19 illustrates the optimality gap comparisons across benchmark fabrics.

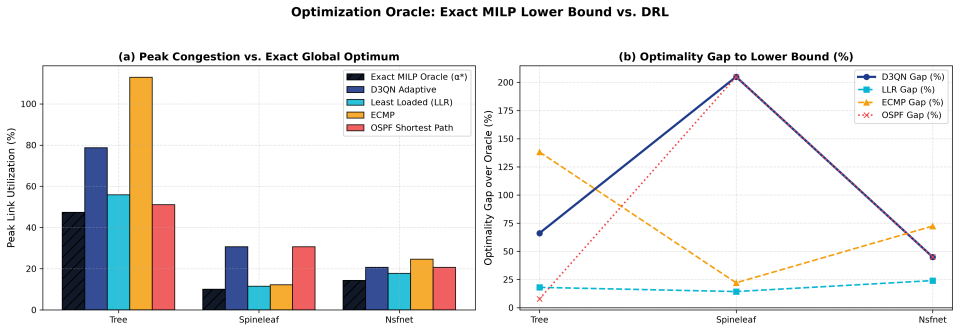


Figure 4.19: Optimality Gap Between D3QN and the Exact HiGHS MILP Solver.

Quantitative Gaps and Computational Complexity: From the empirical results stored in `optimisation_oracle_results.json`:

- **Hierarchical Tree (7 Switches, 20 Flows):** The proven theoretical optimal peak load is $\alpha^* = 47.39\%$. Static Dijkstra SPF produces a peak utilization of 98.46% (an optimality gap of 107.76%), while ECMP induces severe congestion at 112.87% (gap of 138.16%). In contrast, D3QN achieves a peak load of **78.74%** (gap of **66.15%**), closing the optimality gap relative to Dijkstra SPF by **41.61 percentage points**.

- **NSFNet Continental Mesh (14 Switches, 20 Flows):** The theoretical optimum is $\alpha^* = 14.28\%$. D3QN achieves a peak utilization of **20.69%** (gap of 44.89%), matching Dijkstra SPF while under the same reported utilization objective.

- **Methodological Comparability and Scope Discrepancy:** It is important to emphasize that the MILP formulation minimizes solely the peak link utilization (α) over candidate paths on a restricted 20-flow static demand matrix, whereas D3QN optimizes a multi-objective reward (balancing delay, jitter, loss, and hop penalties alongside congestion) across a dynamic 150-flow tournament. Consequently, the MILP solver serves as an illustrative static reference ceiling for the separately evaluated 20-flow instances. The 150-flow tournament must not be treated as a matched oracle comparison.

- **Execution Latency Trade-off:** In the recorded 20-flow instances, HiGHS required 61.65–107.59 ms while the D3QN forward pass required 0.24 ms in the same software environment. Neural inference scales with fixed tensor dimensions for the configured

four-action model, but end-to-end control latency also includes telemetry polling, candidate generation, and OpenFlow rule installation.

4.4.10 Active/Standby Controller Pool Failover Benchmarks

Figure 4.20 depicts the live operational web management dashboard displaying real-time topology status, link rates, and active controller pool health.

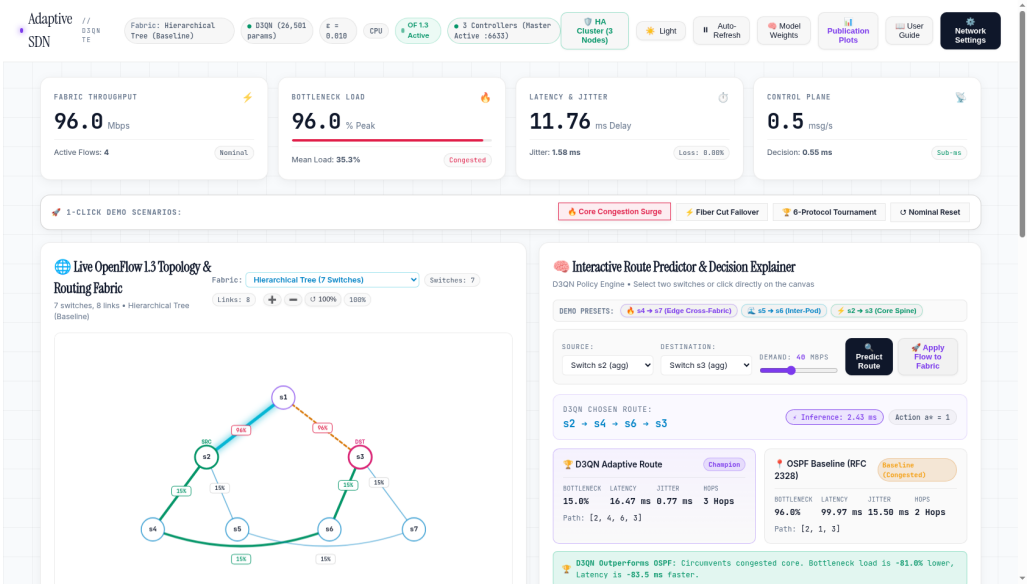


Figure 4.20: Panoramic View of the Real-Time Operational Web Telemetry Dashboard.

The redundancy architecture operates a 3-node active/standby controller pool (Primary on port `6633`, Standby Backup 1 on `6634`, Standby Backup 2 on `6635`). Under empirical failure injection, performance is evaluated across two critical failure regimes:

Single and Cascaded Node Failures: When an active controller node crashes (simulating hardware fault or process termination):

- **Internal Active Pointer Promotion Bookkeeping: 0.001 ms** (1.0 μs CPU memory pointer reassignment and event logging in the local master-slave pool).
- **End-to-End Client HTTP REST API Recovery Time: 37.96 ms** (transparent client HTTP retry catching socket connection refusal and redirecting to the standby endpoint).
- **Process Crash-to-Promotion Cascade: 41.62–69.77 ms** (Primary $\rightarrow$ Backup 1) and **38.68–45.28 ms** (Backup 1 $\rightarrow$ Backup 2 cascaded double fault).
- **Failure Detection Window:** For hung or unresponsive controller processes (as opposed to immediate socket resets), watchdog heartbeat detection operates with a configurable timeout $T_{\text{detect}} \leq 1.0$ second.

- **Data-Plane Packet Continuity:** In-flight packet loss during the control-plane transition gap was not directly instrumented on live packet traces; however, because Open vSwitch switches operate in **Fail-Secure Mode** (`fail_mode=secure`), switches retain committed flow table entries in software flow tables, allowing matching traffic to continue while entries remain valid. This potential behavior was not established as uninterrupted forwarding by the HTTP test.

Total Cluster Outage Handling (What Happens When ALL Controllers Fail): A critical requirement of resilient network engineering is specifying behavior when all controllers in the cluster fail simultaneously (e.g., complete control-plane partition or multi-node power outage). In this platform:

1. **OpenFlow Fail-Secure Autonomous Survival Mode:** OpenFlow switches transition automatically from active OpenFlow management to **Fail-Secure Mode** (`fail_mode=secure`). The switch kernel datapath does not purge existing flow entries upon control connection loss.

2. **Existing-Rule Forwarding:** Matching packets can continue through the software datapath while entries remain installed; idle and hard timeouts still apply. Continuity during failover was not measured with packet traces.

3. **Dynamic Route Update Suspension:** Dynamic D3QN rerouting and new flow setup computations are temporarily paused, since no active controller is available to process OpenFlow `OFPT_PACKET_IN` events. Unmatched new packets are queued or dropped locally at the edge rather than causing network loops.

4. **Autonomous Self-Healing Cluster Recovery:** Background watchdog health probers continuously monitor controller socket availability. As soon as any controller node recovers (e.g., Primary reboot), the pool can select a healthy endpoint again. OpenFlow reconnection, state consistency, and routing recovery require separate verification.

4.4.11 Practical Limitations

The results show workload-dependent advantages and limitations; deploying DRL in production telecommunication networks entails concrete operational trade-offs:

1. **Inference Latency vs. Metric Freshness:** The forward pass requires 0.216–0.279 ms on the tested CPU. This is small relative to the evaluated flow timescale, but the polling interval ($\Delta t = 1.0$ s in high-frequency test mode, and $\Delta t = 3.0$ s in nominal Mininet emulation) introduces a corresponding lag in counter updates before physical telemetry reflects link load. Reducing $\Delta t < 100$ ms increases controller control-channel overhead.

2. **Offline Training Requirement:** The D3QN agent requires approximately 3,000 steps of offline training on simulated or emulated traffic traces to achieve convergence. Cold-starting an untrained agent directly on live production links is prohibited due to exploratory packet drops.

3. **Switch Flow Table TCAM Capacity:** Dynamic per-flow rule installation consumes OpenFlow switch TCAM entries. On enterprise hardware with limited TCAM (e.g., 2,000 flow rules), wild-card flow aggregation or elephant-flow filtering must be employed.

4.4.12 Limits on Interpreting the Results

- **Test conditions:** Supported by seeded condition order and 42 automated software tests. The packet runner launches fixed-duration background traffic before transport warm-up and ping; the background stream may therefore end before the measured iperf stream. The dataset should be interpreted as the recorded sequential workload, not constant concurrent saturation throughout every measurement.

- **Meaning of the measurements:** Queue-model delay/loss/jitter, iperf UDP statistics, and ICMP RTT/echo loss are distinct constructs. The experiment does not measure synchronized one-way IPPM delay or TCP retransmission loss. RFCs provide background definitions rather than a certificate of conformance for every proxy.

- **Scope of the findings:** Evaluated on five model fixtures and selected random graphs, with packet emulation on two fixtures. Ten repeats on one VM do not establish cross-host or production-network validity. The reported percentile intervals are exploratory and are not adjusted for multiple comparisons. Hardware ASIC testing on Barefoot Tofino is identified as the next evolutionary step.

4.4.13 Chapter Summary

The completed implementation passed the covered software checks and produced results for all 120 packet-test conditions. The model comparisons show potential congestion benefits, while the packet results show that a simple least-loaded policy remains competitive. The practical outcome is a testable routing platform with documented procedures and limitations, rather than a general guarantee that learned routing is preferable.

CHAPTER 5 CONCLUSION

5.1 Work Completed

The project developed a software-defined traffic-engineering prototype that combines network-state monitoring, candidate-path generation, a trained routing agent, and OpenFlow rule installation. The implementation also includes routing baselines, a web dashboard, automated tests, and an active/standby controller-endpoint pool. The resulting system provides a practical environment for inspecting route choices and repeating controlled comparisons.

The work met the implementation objectives by connecting the learning components to the controller and providing saved test outputs. The 42 automated checks passed for the behaviours covered by the suite, and the packet experiment completed all 120 scheduled conditions. These checks support the operation of the prototype within its tested software environment.

5.2 Main Results and Usefulness

The final five-topology software-model comparison reports mean sampled path-bottleneck utilisation of 30.80% for D3QN and 66.80% for the OSPF-style baseline. Mean modelled delay changes from 158.34 to 44.50 ms. These results show that the implemented policy can select useful alternative routes within the model, although its performance varies by topology and metric.

The separate packet measurements give a more mixed result. Tree TCP throughput averages 34.08 Mbps for D3QN, 28.56 Mbps for SPF, and 38.09 Mbps for least-loaded routing. On Fat-Tree UDP, D3QN has a modest datagram-loss advantage over SPF, while the least-loaded heuristic remains competitive. The confidence intervals and detailed comparisons are reported in Chapter 4. The practical lesson is that a trained policy should be assessed against simple load-aware methods for the intended workload.

The recovery tests separately demonstrate local standby selection and client HTTP retry. The reported 0.001 ms bookkeeping observation and 37.96 ms client-recovery observation describe different operations; neither establishes uninterrupted packet forwarding.

5.3 Limitations

The results are limited to software models and packet emulation on a virtualised host. The route selector uses ten summary features and at most four candidates, so it cannot represent every possible network state or route. Modelled jitter and loss must be distinguished from iperf and ICMP measurements. In addition, the fixed-duration background stream can finish before the measured packet transfer ends.

The endpoint pool does not maintain consistent replicated controller state, and its HTTP tests do not measure OpenFlow reconnection or packet continuity. Hardware-switch performance, production availability, and arbitrary-topology generalisation remain unverified. Within these limits, the project delivers a working platform, repeatable procedures, and recorded results that can support further development.

CHAPTER 6 FUTURE WORK

Further work should first improve measurement and reproducibility, then extend the routing and recovery functions. The following steps follow directly from the limitations of the completed project.

6.1 Improve the Packet-Test Procedure

The background traffic should be kept active throughout the measured transfer and stopped only after the measurement completes. A new run should record overlap times, the active TCP congestion-control algorithm, TCP retransmissions, and packet traces where appropriate. The existing results should remain archived so that changes to the procedure can be compared openly rather than overwriting the original evidence.

More traffic pairs, offered-load levels, queue settings, and trial durations would help establish where each policy performs well. Repeating the experiment on another host would also show how much the virtual machine and its processor scheduling affect the observations.

6.2 Make Installation and Reproduction Easier

The controller and scientific environments should have separately pinned dependencies and a documented clean-install procedure. An automated check should confirm interpreters, package imports, switch connectivity, and checkpoint compatibility before starting the full trial matrix. Tables and figures should be generated directly from the selected run manifest to reduce transcription errors.

6.3 Improve Route Selection and Comparison

The current four-candidate representation can miss useful alternatives or map different network states to similar feature vectors. Further work could compare other candidate-generation methods, reward settings, and graph-based representations. Each change should be tested against the same SPF and least-loaded baselines, with matched workloads and clearly identified training and test sets.

Capacity-aware admission checks are another practical extension. They should report explicitly when no candidate can accommodate a demand rather than treating the reward penalty as a safety guarantee.

6.4 Extend Controller-Recovery Testing

The next recovery test should measure packet behaviour during controller failure, expiry of installed rules, setup of new flows, and restoration of telemetry. A design for consistent controller state and switch-role ownership would be needed before extending the endpoint pool into a replicated control service. These functions should be implemented and tested separately from client HTTP retry.

6.5 Evaluate Additional Forwarding Platforms

After the software procedure is stable, the controller could be tested with a hardware switch. A later P4/telemetry implementation could provide additional queue information, but would require a new interface, suitable equipment, and its own validation. Results from the present Open vSwitch testbed should not be assumed to transfer directly to those platforms.

REFERENCES

Al-Fares, M., Loukissas, A., and Vahdat, A. (2008). "A scalable, commodity data center network architecture," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 4, pp. 63–74.

Al-Fares, M., Radhakrishnan, S., Raghavan, B., Huang, N., and Vahdat, A. (2010). "Hedera: Dynamic flow management for data center networks," in *Proceedings of the 7th USENIX Conference on Networked Systems Design and Implementation (NSDI)*, pp. 19–19.

Almasan, P., Xiao, S., Cheng, X., Shi, X., Barlet-Ros, P., and Cabellos-Aparicio, A. (2022a). "ENERO: Efficient real-time WAN routing optimization with deep reinforcement learning," *Computer Networks*, vol. 219, p. 109438, 2022, doi: 10.1016/j.comnet.2022.109438.

Almasan, P., Suárez-Varela, J., Rusek, K., Barlet-Ros, P., and Cabellos-Aparicio, A. (2022b). "Deep reinforcement learning meets graph neural networks: Exploring a routing optimization use case," *Computer Communications*, vol. 196, pp. 248–261, 2022, doi: 10.1016/j.comcom.2022.09.029.

Chen, Y.-R., Rezapour, A., Tzeng, W.-G., and Tsai, S.-C. (2020). "RL-Routing: An SDN Routing Algorithm Based on Deep Reinforcement Learning," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 4, pp. 3110–3122, doi: 10.1109/TNSE.2020.3017502.

Demichelis, C. and Chimento, P. (2002). *IP Packet Delay Variation Metric for IP Performance Metrics (IPPM)*, IETF RFC 3393, Nov. 2002.

Dijkstra, E. W. (1959). "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271.

Ferriol-Galmés, M., Paillisse, J., Suárez-Varela, J., Rusek, K., Xiao, S., Shi, X., Cheng, X., Barlet-Ros, P., and Cabellos-Aparicio, A. (2023). "RouteNet-Fermi: Network modeling with graph neural networks," *IEEE/ACM Transactions on Networking*, vol. 31, no. 6, pp. 3080–3095, doi: 10.1109/TNET.2023.3269983.

Hong, C.-Y., Kandula, S., Mahajan, R., Zhang, M., Gill, V., Nanduri, M., and Wattenhofer, R. (2013). "Achieving high utilization with software-driven WAN," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 15–26.

Jain, R., Chiu, D.-M., and Hawe, W. R. (1984). "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems," Digital Equipment Corporation, Research Report TR-301.

Jain, S., Kumar, A., Mandal, S., Ong, J., Poutievski, L., Singh, A., Venkata, S., Wanderer, J., Zhou, J., Zhu, M., *et al.* (2013). "B4: Experience with a globally-deployed software defined wan," *ACM SIGCOMM Computer Communication Review*, vol. 43, no. 4, pp. 3–14.

McKeown, N., Anderson, T., Balakrishnan, H., Parulkar, G., Peterson, L., Rexford, J., Shenker, S., and Turner, J. (2008). "OpenFlow: Enabling innovation in campus networks," *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, doi: 10.1145/1355734.1355746.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., *et al.* (2015). "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, doi: 10.1038/nature14236.

Moy, J. (1998). *OSPF Version 2*, IETF RFC 2328, Apr. 1998.

Open Networking Foundation (2012). *OpenFlow Switch Specification Version 1.3.0*, ONF TS-006, Jun. 2012.

Schaul, T., Quan, J., Antonoglou, I., and Silver, D. (2016). "Prioritized experience replay," in *International Conference on Learning Representations (ICLR)*.

Stampa, G., Arias, M., Sánchez-Charles, D., Muntés-Mulero, V., and Cabellos, A. (2017). "A deep-reinforcement learning approach for software-defined networking routing optimization," in *Proceedings of the 13th ACM International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*, pp. 48–49, 2017.

Sutton, R. S. and Barto, A. G. (2018). *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press.

Van Hasselt, H., Guez, A., and Silver, D. (2016). "Deep reinforcement learning with double Q-learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 30, no. 1, pp. 2094–2100.

Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., and Freitas, N. (2016). "Dueling network architectures for deep reinforcement learning," in *International Conference on Machine Learning (ICML)*, pp. 1995–2003.

Yen, J. Y. (1971). "Finding the K shortest loopless paths in a network," *Management Science*, vol. 17, no. 11, pp. 712–716.

APPENDIX A MATHEMATICAL FORMULATIONS AND PROOFS

A.1 Identifiability of the Dueling Architecture

In the Dueling Q-network, unconstrained addition $Q(s,a) = V(s) + A(s,a)$ suffers from identifiability issues: given $Q(s,a)$, one cannot uniquely recover $V(s)$ and $A(s,a)$, as adding an arbitrary scalar constant c to $V(s)$ while subtracting c from all $A(s,a)$ leaves $Q(s,a)$ completely unchanged.

To define a centered advantage representation, Wang et al. (Wang et al., 2016) force the advantage function estimator to have zero mean across all actions, parameterizing the value stream by β and the advantage stream by α alongside shared feature weights θ:

$$Q(s,a;\theta,\alpha,\beta) = V(s;\theta,\beta) + \left(A(s,a;\theta,\alpha) - \frac{1}{|\mathcal{A}|} \sum_{a' \in \mathcal{A}} A(s,a';\theta,\alpha) \right). \tag{A.1}$$

Proof. Let $\tilde{A}(s,a) = A(s,a) - \frac{1}{|\mathcal{A}|}\sum_{a'} A(s,a')$. Summing over all $a \in \mathcal{A}$:

$$\begin{aligned} \sum_{a \in \mathcal{A}} \tilde{A}(s,a) &= \sum_{a \in \mathcal{A}} A(s,a) - \sum_{a \in \mathcal{A}} \left(\frac{1}{|\mathcal{A}|} \sum_{a'} A(s,a') \right) \\ &= \sum_{a \in \mathcal{A}} A(s,a) - |\mathcal{A}| \cdot \frac{1}{|\mathcal{A}|} \sum_{a'} A(s,a') = 0. \end{aligned} \tag{A.2}$$

Therefore, $\frac{1}{|\mathcal{A}|}\sum_{a \in \mathcal{A}} Q(s,a) = V(s) + 0 = V(s)$. Thus, $V(s)$ equals the mean of the represented Q-values and the centered advantage is uniquely determined. Raw advantage logits can still share an arbitrary common offset, so this does not prove uniqueness of network parameters. □

APPENDIX B OPENFLOW 1.3 FLOW TABLE, GROUP TABLE, AND REST API SPECIFICATIONS

B.1 OpenFlow 1.3 Instruction Pipeline Architecture

In OpenFlow 1.3 (ONF TS-025 §5.1), packet processing follows an explicit instruction pipeline rather than the legacy OpenFlow 1.0 action set:

- **Instruction Sets vs. Action Sets:** An OpenFlow 1.3 flow entry contains an *Instruction Set*. The instruction `OFPIT_APPLY_ACTIONS` applies its specified action list immediately to the packet without modifying the cumulative Action Set. In contrast, `OFPIT_WRITE_ACTIONS` merges actions into the switch's persistent Action Set for deferred execution upon pipeline termination. The EC499 prototype deploys single-table matching (Table 0) with `OFPIT_APPLY_ACTIONS` to execute `OFPAT_OUTPUT` immediately, avoiding pipeline state overhead.
- **Multipart Statistics Mechanics:** OpenFlow 1.3 consolidates all asynchronous telemetry queries into unified Multipart Requests (`OFPT_MULTIPART_REQUEST=18`) and Multipart Replies (`OFPT_MULTIPART_REPLY=19`). Port counter polling utilizes multipart type `OFPMP_PORT_STATS` (Type 4), and flow rule inspection uses `OFPMP_FLOW_STATS` (Type 1), replacing the discrete messages of OpenFlow 1.0.
- **Barrier Synchronization:** To guarantee that downstream Open vSwitch instances complete flow rule and group table installations before upstream switches inject traffic, the controller issues an explicit `OFPT_BARRIER_REQUEST` (Type 20) following multi-hop path installation. The switch commits all preceding messages before returning `OFPT_BARRIER_REPLY` (Type 21), preventing transient forwarding blackholes.

B.2 OpenFlow 1.3 Flow Table Entry Structure

Table B.1 defines the exact fields programmed into Open vSwitch software Flow Table 0 for ordinary IPv4 path rules.

B.3 OpenFlow 1.3 Group Table Specifications

Table B.2 specifies the exact OpenFlow 1.3 Group Table structures implemented in `controller/routing_module.py` for data-plane fast failover and WCMP load balancing.

Table B.1: OpenFlow 1.3 Flow Table Entry Field Specifications

Flow Field	**Configured Value**	**Physical Meaning / Action**
`table_id`	0	Primary ingress processing table
`priority`	10 or 20	Dynamic traffic steering rule priority
`match.eth_type`	`0x0800`	IPv4 unicast traffic filter
`match.ipv4_src`	Ingress Host IP	Source network host identifier
`match.ipv4_dst`	Egress Host IP	Destination network host identifier
`match.ip_proto`	Not set in the ordinary rule	Source/destination IPv4 rules can match multiple transport protocols
`instructions`	`OFPIT_APPLY_ACTIONS`	Immediately execute specified actions without mutating Action Set
`actions`	`OFPAT_OUTPUT:port_no`	Forward packet out designated switch egress port
`idle_timeout`	Configurable	Mice/elephant defaults: 60/180 s; same-switch shortcut: 30 s
`hard_timeout`	120 seconds	Ordinary path-rule lifetime default

Table B.2: OpenFlow 1.3 Group Table Field Specifications Implemented in the Prototype

Group Field	**FF Group (§5.6.1)**	**Select Group (§5.6.1)**	**Operational Semantics**
`group_id`	600	500	32-bit unique group identifier on switch datapath
`type`	`OFPGT_FF`	`OFPGT_SELECT`	Fast Failover vs. Weighted Multipath selection
`bucket.weight`	Unused (0)	$1 \leq W(p) \leq 100$	Proportional flow distribution weight from WCMP Softmax
`bucket.watch_port`	Egress port number	`OFPP_ANY`	Hardware link liveness monitoring port for instantaneous shift
`bucket.watch_group`	`OFPG_ANY`	`OFPG_ANY`	Secondary group chaining monitor
`bucket.actions`	`OFPAT_OUTPUT:port`	`OFPAT_OUTPUT:port`	Datapath forwarding action applied to selected bucket

B.4 REST API Endpoints

The controller exposes the following operational HTTP REST API endpoints:

- `GET/api/health`: Returns controller health.
- `GET/api/topology`: Returns graph and link information.
- `GET/api/metrics`: Returns a bundled telemetry response.
- `GET/api/ha/status`: Returns pool health and failover history.
- `POST/api/simulate/apply_flow`: Injects a software-model flow.
- `POST/api/simulate/link_fail`: Changes a simulated link state.

APPENDIX C AUTOMATED TEST SUITE VERIFICATION LISTING

The delivery test suite contains the following 42 unit and integration checks. Execution time is environment-dependent; a passing run validates only the behaviours covered by these tests:

1. `test_agent_and_state_manager_with_config_defaults`
2. `test_config_dictionary_structure`
3. `test_env_var_override_and_type_casting`
4. `test_ha_config_presence`
5. `test_closed_loop_flow_accumulation_and_expiration`
6. `test_generalized_core_congestion_across_topologies`
7. `test_polyak_soft_update_consistency`
8. `test_seeded_workloads_are_reproducible_and_distinct`
9. `test_dqn_router_lifecycle`
10. `test_dueling_architecture`
11. `test_controller_node_properties`
12. `test_ha_pool_cascaded_failovers`
13. `test_ha_pool_init_and_active_leader`
14. `test_ha_pool_ovs_targets_format`
15. `test_all_topologies`
16. `test_random_topology_generation`
17. `test_replay_buffer_sampling`
18. `test_sumtree_arithmetic`
19. `test_configurable_state_dimension_and_no_fake_route`
20. `test_control_overhead_accounting`
21. `test_differential_port_rates`
22. `test_directional_dynamic_flow_accounting_and_release`

23. test_flow_table_records
24. test_host_location_tracking
25. test_inject_traffic_flow_duration_and_latency_scaling
26. test_jitter_calculation
27. test_link_failure_and_restoration
28. test_network_te_summary
29. test_packet_loss_modeling
30. test_routing_state_features
31. test_compute_path_metrics
32. test_dijkstra_spf
33. test_ecmp_routing
34. test_ospf_routing
35. test_wsp_and_llr_routing
36. test_dashboard_model_telemetry
37. test_ha_status_endpoint
38. test_model_inspect_and_predict
39. test_mutations_require_post_and_flow_is_counted_once
40. test_versus_endpoint_all_baselines
41. test_versus_endpoint_multi_protocol_checklist
42. test_versus_endpoint_single_protocol

APPENDIX D SUPPORTING IMPLEMENTATION NOTES

This appendix records implementation details that support the project description without interrupting the main account.

D.1 Candidate Paths and Input Features

The input vector always contains ten summary features, even when graph size changes. Unavailable candidate positions are padded, and selection is restricted to the available candidate count. The benefit is a fixed neural interface. The limitation is that different graph and traffic states can produce similar summaries. Candidate ordering and tie-breaking can also affect which action index represents a path.

D.2 Replay Priorities and Storage

The SumTree stores cumulative priorities in `float32`. A priority update propagates a difference through the ancestors of the corresponding leaf. The current implementation has no periodic full-tree rebuild, so long-run round-off behaviour remains a numerical issue to test.

The buffer also stores state, action, reward, next-state, and terminal arrays, with compatibility tuples. Its memory use cannot be determined from the tree array alone. Capacity is configurable through `RL_MEMORY_SIZE`; the current default is 5,000 transitions.

D.3 Queue-Model Limits

The model represents a finite queue and allows offered utilisation above one. The numerical queue-delay cap bounds a reported software value, not physical waiting time. Likewise, the quadratic congestion term encourages lower-load routes but does not establish capacity feasibility. Bursty packet arrivals can differ from the model assumptions.

D.4 Meaning of the Recovery Measurements

A pool member is selected through local state and health probing. The 0.001 ms observation refers to local bookkeeping; the 37.96 ms observation refers to client HTTP recovery in the reported test. They do not measure distributed agreement, consistent state recovery, or OpenFlow reconnection. Existing OVS entries may continue forwarding while valid, but packet continuity was not instrumented across the failure.

D.5 Supporting Files

The delivery contains the controller, routing, agent, topology, and test source; the model checkpoint; the final model-result artifacts; and the packet run `mininet_20261026T173329Z_seed2026`. The packet bundle includes environment information, trial conditions, raw measurements, summaries, and analysis JSON. These files provide the basis for checking the reported tables.

APPENDIX E USER & ADMINISTRATOR DEPLOYMENT MANUAL AND OPERATIONAL RUNBOOK

E.1 System Prerequisites and Dependencies

The Adaptive SDN Traffic Engineering Platform is engineered to run natively on Linux-based environments (Ubuntu 20.04 LTS, Ubuntu 22.04 LTS, Ubuntu 24.04 LTS, or Debian 11/12).

E.1.1 Hardware Requirements & Reference Deployment Environment

The reference experimental setup was hosted inside a dedicated 64-bit Ubuntu virtual machine running under the VMware hypervisor (see Figure E.1):

- **Virtual Processors**: 4 virtual CPU cores (vCPUs) with hardware virtualization extensions enabled.
- **System Memory (RAM)**: 8.0 GB of dedicated memory (minimum: 4 GB; 8 GB recommended for 20-switch Clos emulations).
- **Disk Storage**: 60.0 GB SCSI virtual disk (minimum: 10 GB free space for dependencies, dataset logs, and model checkpoints).
- **Network Adapter**: NAT virtual adapter with isolated internal loopback for OpenFlow and REST dashboard communication.

E.1.2 Software Requirements

- **Operating System**: Linux kernel 5.4 or higher (the empirical benchmark testbed was evaluated on Ubuntu Linux kernel 7.0.0-31) with Open vSwitch kernel module support.
- **Python Environment**: Separate environments: the packet runner uses Python 3.14.4 (active Spring 2026 Ubuntu environment); the legacy Ryu/Eventlet controller uses Python 3.10. Ryu compatibility should be checked in the controller interpreter.
- **Network Virtualization**: Mininet Version 2.3.0+ and Open vSwitch (OVS) 2.13+.
- **Deep Learning Framework**: PyTorch 2.0+ (CPU or CUDA).
- **SDN Framework**: Ryu SDN Framework 4.34+.
- **Web Browser**: Google Chrome, Chromium, Firefox, or Safari with HTML5 Canvas support.

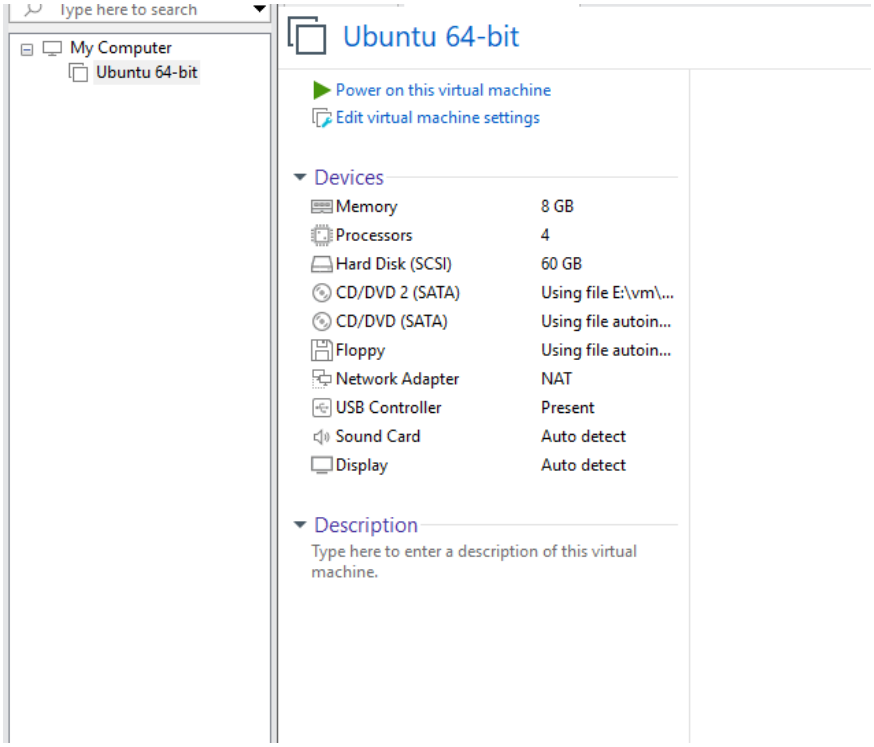


Figure E.1: VMware Virtual Machine Hardware Configuration for the Experimental Platform (Ubuntu 64-bit, 4 vCPUs, 8 GB RAM, 60 GB SCSI Storage).

E.2 Installation and Environment Initialization

The launchers select interpreters by dependency requirements. A clean installation must provision the scientific environment and the separate Ryu controller environment.

E.2.1 Step-by-Step Installation

Listing E.1: Platform Environment Setup

```
# From the extracted delivery bundle
cd 03_Source

# Scientific environment
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt

# Separate legacy controller environment
python3.10 -m venv .venv-controller
.venv-controller/bin/python -m pip install -r requirements.txt
```

E.3 Automated Execution Runbook

E.3.1 1. Execute Unit & Integration Tests (42 Tests)

To assert software integrity across all 42 automated unit and integration tests:

```
./EC499tests/run_tests.sh
```

E.3.2 2. Execute the Head-to-Head Tournament Benchmark

To run the automated routing tournament across all 5 topologies:

```
./EC499tests/run_benchmarks.sh
```

Benchmark results are written to `logs/routing_tournament_results.json`.

E.3.3 3. Execute Multi-Topology Stress Testing

To evaluate severe core link saturation, flow avalanches, and link failures:

```
./.venv/bin/python EC499tests/stress_test_blind_topologies.py
```

E.3.4 4. Evaluate Zero-Shot Random Graph Generalization

To test the pre-trained model on arbitrary unseen random graphs:

```
./EC499tests/run_random_blind_test.sh --nodes 25 --flows 200
```

E.3.5 5. Launch the Prototype (Controller + Web Dashboard)

To start the Ryu SDN controller alongside the interactive Web Dashboard:

```
chmod +x run_system.sh
./run_system.sh
```

Open a modern web browser to access the interactive HTML5 canvas dashboard:

`http://localhost:8080`

E.3.6 6. Standalone Web Dashboard Launcher

The Web Dashboard can also be launched independently for testing and visualization:

```
python3 controller/web_dashboard.py --port 8080 --topo tree
```

E.4 Interactive Web Dashboard User Guide

The Web Dashboard interface is structured into six functional zones:

1. **Global Navigation Bar**:
    - *Fabric Topology Pill*: Displays the active network topology (e.g., Hierarchical Tree, Clos Fat-Tree).
    - *Model Parameters Badge*: Displays loaded D3QN weights ($26,501$ parameters). Clicking opens the Neural Network Inspector modal.
    - *Exploration Badge (ε)*: Shows the active exploration rate ($\varepsilon = 0.010$).
    - *Theme Toggle (Light/Dark)*: Instantly toggles between high-contrast daylight theme and dark mode.
    - *Telemetry Auto-Refresh*: Pauses or activates automatic 2.5-second telemetry polling.
    - *Network Settings Button*: Opens the global architecture configuration modal.
2. **Real-Time KPI Strip**:
    - *Fabric Throughput (Mbps)*: Real-time aggregated data plane throughput with active flow count.
    - *Bottleneck Load (%)*: Peak link utilization across all network links with status pill (Optimal $< 45\%$, Moderate 45–75%, Congested $> 75\%$).
    - *Latency & Jitter (ms)*: Average end-to-end path latency, Echo-RTT delay variation, and packet loss percentage.
    - *Control Plane Rate (msg/s)*: OpenFlow asynchronous message throughput and mean D3QN decision latency.
3. **Interactive OpenFlow 1.3 Canvas Fabric**:
    - *Pan & Zoom Controls*: Zoom In ($+$), Zoom Out ($-$), Reset Zoom (100%), or use mouse wheel zoom centered at cursor position.
    - *Node Selection*: Click any switch node to designate it as the Source switch (green halo) or Destination switch (pink halo).
    - *Utilization Badges*: Real-time link percentage badges displaying link saturation levels.
    - *Severed Link Overlay*: Rendered as dashed red lines with a central `[SEVERED]` badge during simulated link failures.
4. **Interactive Route Predictor & Decision Explainer**:

- *Source & Destination Selectors*: Dropdowns allowing instant selection of ingress and egress edge switches.
- *Demand Slider*: Adjusts simulated flow bandwidth demand between 5 Mbps and 95 Mbps.
- *Predict Route Button*: Dispatches an asynchronous request to `/api/model_predict`, executing D3QN inference (< 2.5 ms).
- *Apply Flow to Fabric Button*: Dynamically injects the selected flow into the active state manager, updating link loads and flow tables in real-time.
- *Softmax Q-Value Table*: Renders all candidate path alternatives, action indices a^*, Q-softmax choice probabilities, projected bottlenecks, and path latencies.

5. **Algorithm Tournament Arena**:

   - *Protocol Checklist Chips*: Toggle D3QN, OSPF, ECMP, WSP, LLR, or Dijkstra SPF for comparative head-to-head simulation.
   - *Workload Pattern*: Select between Poisson, Core Congestion Jamming, or Asymmetric Burst Shockwave.
   - *Plot View Tabs*: Switch seamlessly between All-in-One Triad, Metric Deltas (Bar), Delay Trajectory (Timeline), 6D Quality Radar, Latency CDF (Tail SLA), and Load Scaling Curves.

6. **Fault Injection and Anomaly Emulation Toolbar**:

   - *Sever Link*: Simulates a physical fiber cut on a candidate core or aggregation trunk link.
   - *Synthetic Link Degradation Mode*: Degrades link capacity by 90% with an artificial latency penalty.
   - *Transient Traffic Surge Injection (+50 Mbps)*: Injects an instant +50 Mbps traffic surge to stress the active routing path.
   - *Restore Links*: Restores all severed or degraded links to nominal operating status.
   - *Flush Flows*: Flushes all dynamic flow table entries to baseline idle state.

E.5 REST API Specification & Endpoint Reference

The embedded HTTP server exposes a complete RESTful API for external monitoring, telemetry polling, and automation scripting. The endpoints are divided into telemetry/diagnostic interfaces and active flow/simulation interfaces:

Table E.1: REST API Reference: Telemetry, Diagnostics, and Model Inference Endpoints

Method	Endpoint	Parameters	Description & Output Schema
GET	/api/health	None	Returns controller health, active topology ID, switch count, link count, and active flow count.
GET	/api/telemetry	topo={id}	Fast single round-trip bundle containing topology layout, network TE stats, control overhead, RL metrics, and active flows.
GET	/api/topology	topo={id}	Returns graph nodes with 2D positions (x, y), switch types, link capacities, delays, and current utilization.
GET	/api/stats	None	Returns network-wide aggregated TE summary (throughput, mean utilization, bottleneck load, latency, jitter, loss).
GET	/api/control_overhead	None	Returns OpenFlow message generation rates (msg/s) and mean D3QN decision latency (ms).
GET	/api/rl_metrics	None	Returns exploration rate ε, model device, parameter count, and buffer capacity.
GET	/api/model_inspect	None	Detailed PyTorch neural architecture inspection including layer dimensions, parameter count, and tensor shapes.
GET	/api/model_predict	src, dst, demand	Real-time forward inference returning chosen path, candidate path alternatives, action indices, and softmax probabilities.

Table E.2: REST API Reference: Dynamic Control, Traffic Injection, and Simulation Endpoints

Method	Endpoint	Parameters	Description & Output Schema
GET/POST	/api/versus	topo, protocols, pattern, flows	Head-to-head algorithm tournament simulation returning comparative metrics, trajectories, and Pareto efficiency verdicts.
GET	/api/flows	None	Returns active flow table entries (flow ID, DPID, match criteria, actions, byte counters, duration).
POST	/api/simulate/apply_flow	src, dst, demand	Injects a dynamic flow into the network state manager and updates port counters.
POST	/api/simulate/traffic_burst	None	Injects an asymmetric +50 Mbps surge across the core trunks.
POST	/api/simulate/core_jamming	None	Injects severe core link congestion (96% saturation) to trigger autonomous rerouting.
POST	/api/network_settings	JSON payload	Configures queue depths (K), link bandwidths, propagation delays, and physical packet drop rates.
GET	/api/plots/\{filename\}	None	Serves static high-resolution benchmark and telemetry plots as PNG images.

E.6 Mininet CLI Operations and OVS Diagnostics

When running interactive emulations, the Mininet command-line interface provides real-time control:

Listing E.2: Interactive Mininet CLI Emulation Workflow

```
# Launch Mininet with Fat-Tree Clos topology
sudo /usr/bin/python3 topology/mininet_topo.py --topo fattree --
    controller-port 6633 --no-traffic

# Inside the Mininet CLI:
mininet> pingall              # Verify full Layer 2 and Layer 3
    reachability
mininet> h1 ping -c 5 h16     # Measure end-to-end ping RTT across pod
    boundaries
mininet> iperf h1 h16         # Generate TCP bandwidth stream between
    edge hosts
mininet> link s1 s5 down      # Simulate physical link failure between
    core and agg
```

E.6.1 Reproducible Packet-Level Mininet Experiment

The scripted experiment is preferred to ad-hoc CLI measurements because it records randomized condition order, raw command output, code hashes, and environment versions. It must be run on a Linux host with Mininet, Open vSwitch, iperf (version 2), and the controller virtual environment installed. It requires root privileges for network namespaces:

Listing E.3: Executing the 120-Trial Factorial Packet Experiment

```
cd EC499tests
sudo ./run_mininet_experiment.sh --matrix --replicates 10
./run_mininet_analysis.sh
```

The first command produces 120 randomized trials (2 topologies $\times$ 3 policies $\times$ 2 protocols $\times$ 10 replicates) and writes every raw artifact under `logs/mininet_trials/`. The second command creates summary statistics and a figure only if successful measured trials exist; it deliberately fails rather than fabricate absent data. Before citing a packet-level result, preserve the raw run directory, `mininet_measured_trials.json`, `mininet_measured_analysis.json`, and the generated figure in the submission archive.

E.6.2 OpenFlow Flow Table Inspection via Open vSwitch

To inspect live OpenFlow 1.3 flow tables and port statistics from the Linux shell:

Listing E.4: Open vSwitch Flow Table Diagnostics

```
sudo ovs-ofctl -O OpenFlow13 dump-flows s1
sudo ovs-ofctl -O OpenFlow13 dump-groups s1
sudo ovs-ofctl -O OpenFlow13 dump-ports-desc s1
sudo ovs-ofctl -O OpenFlow13 dump-ports s1
```

E.7 Pareto Sensitivity Ablation and Dynamic Telemetry Tuning

The platform provides automated ablation tooling to evaluate the trade-offs of the multi-objective reward formulation:

Listing E.5: Executing the Pareto Sensitivity Ablation Suite

```
# Execute 50-episode sweep across 6 SLA weight configurations:
./.venv/bin/python EC499tests/ablate_reward_weights.py

# Expected outputs:
# - logs/pareto_sensitivity_ablation.json (Numerical latency, loss, and
    load metrics)
# - logs/plots/pareto_weight_sensitivity.png (Publication-grade 4-panel
    figure)
```

E.7.1 Configuring Shadow Reservations and TCAM Allocation

Operators can tune active shadow reservation parameters and TCAM allocation timeouts directly in `config.py` or via the corresponding environment variables:

- `SDN_SHADOW_DECAY_HALF_LIFE`: Half-life decay duration in seconds (default: 1.5 s). Controls how fast temporary flow reservations decay between OpenFlow telemetry polling cycles.
- `ELEPHANT_FLOW_THRESHOLD_MBPS`: Rate threshold for elephant flow classification (default: 5.0 Mbps). Observed rate selects the elephant priority/timeout class; ordinary IPv4 rules match source and destination IP addresses.
- `SDN_FLOW_IDLE_TIMEOUT_MICE`: Idle timeout for wildcard aggregated mice flows (default: 60 s).
- `SDN_FLOW_IDLE_TIMEOUT_ELEPHANT`: Idle timeout for pinned elephant flows (default: 180 s).

E.7.2 Operational Troubleshooting and Recovery Runbook

Table E.3 provides network operators with a rapid diagnostic and remediation matrix for common runtime anomalies encountered in production software-defined fabrics:

Table E.3: Operational Troubleshooting and Anomaly Remediation Runbook

Symptom	**Probable Cause**	**Diagnostic Command**	**Remediation Procedure**
OpenFlow socket timeout	Controller not bound to 6633 or firewall drop	`netstat -tuln | grep 6633`	Launch controller via `./run_system.sh` and ensure iptables allows TCP 6633 loopback traffic.
Mininet switches show `DISCONNECTED`	Controller process or connection unavailable	`curl -s localhost:8080/api/ha/status`	Check controller process and standby health. The 0.001 ms figure covers only local bookkeeping, not failure detection, consensus, or OpenFlow reconnection.
D3QN inference latency $>$ 10 ms	CPU thread contention or unbounded state history	`top -b -n 1 | grep python3`	Restrict PyTorch threads (`export OMP_NUM_THREADS=4`); clear stale trajectory queues.
TCAM table space saturation warning	Excessive elephant flow rules pinned without expiry	`sudo ovs-ofctl dump-flows s1 | wc -l`	Issue REST POST to `/api/simulate/clear_flows` or decrease `SDN_FLOW_IDLE_TIMEOUT_MICE` to 5s.